ROB

We have seen that we can improve performance a lot by reordering instns. But things are not that rosy in instn reorder land. In real programs we can have exceptions such as div by 0, that can be messed up if we don't execute instns exactly in program order. This lesson talks about how to cleanup reordering mess when exceptions do happen.

Exceptions in OoO processor

	INST	IS	DIS	WR	
1	DIV F10, F0, F6	1	2	42	⇐
2	L.D F2, 45(R3)	2	3	13	✓
3	MUL F0, F2, F4	3	14	19	✓
4	SUB F8, F2, F6	4	14	15	✓

F6 == 0 ⇒ DIVIDE BY ZERO IN CYCLE 40

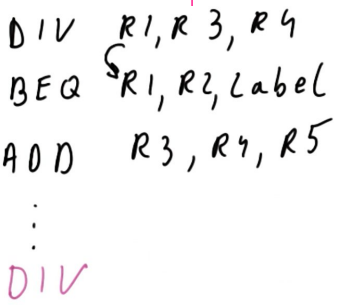
Exceptions in OoO processor : Its part of whats wrong with tomasulo's algorithm as far as modern processors are concerned. Lets look at these 4 instns and how they execute. Say we are able to issue every instn. in subsequent cycles like in 1, 2, 3 & 4. Say Div takes 40 cycles for exe. meanwhile other instns are dispatched and executed, obeying the raw dependencies. Lets say F6 is 0, and we get a div by 0 error, and say that is detected in cycle 40(somewhere towards the end of divide, we find its div by 0), At such an event, what should happen in a processor is ,

* The PC for this instn should be saved,and we should jump to an exception handler, and if the exception handler comes back, we should be back to this instn. and start to execute from here

However what really happens is , in cycle 40 we have already executed and written results for subsequent instn.s , and the div is the only one that hasn't been done, so when we go to exception handler and come back, we will use the F0 produced by mul, and so div can never get a correct result even if we correct F0(to be epsilon which tends to zero).

This can also happen if a load instn has a page fault, if so we need to bring the page from the disk , go back to execution of load, but by that time, instns after the load might have executed and theres no way to proceed correctly. So this problem of handling exceptions precisely, is what considered to be major drawback of tomasulo's algorithm as it was implemented.

Branch misprediction in OoO processor



Another thing that's a problem with tomasulo's is that when we do branch mispred., so how do we recover from a branch mispred? Say we ahve a program like this, we have divide , that takes say 40 cycles, and once we get that result , we can resolve the following branch. So this branch can be predicted, but it can can a very long time to realize if it is mispred. Say we fetch the ADD, (but in actual the branch should have been taken,) and it is done i.e R3 is already written, once we realize that we have mispred this branch, possibly after many cycles, we should behave like we have never executed ADD and should continue executing from the Label that the branch is pointing to , but that becomes impossible because R3 by that time is already written, so we see incorrect value of R3, when we jump to Label. The problem is very similar to precise exceptions, in the sense that, an instn can complete and write to its regs before preceding instns have fully been verified, and before we even know that this instn (ADD here) should have been executed.

A final problem occurs because of so called phantom exceptions. Lets say there is a DIV (in pink) , and say we have mispred this branch, and say DIV generates an exception, the exception will triggered even though this instn was never supposed to be executed . By the time we know this DIV was not to be exe, we have done lot of exception processing.

So somehow we need to take care of not having exceptions until we are sure that they should happen.

So lets see how should we **Correct OoO execution**

* We should execute OoO
* We should broadcast OoO
* But We should deposit values in regs IN-ORDER

The last one is needed because if we deposit values in OoO, and subsequently, we discover that one of the earlier instns shouldn't have been done (because it was some exception or some branch mispred), We will see incorrect value in regs. SO if we deposit in-order this cannot happen. If we deposit in order that means all the previous instns have been finished successfully

So what we need is a structure called reorder buffer, even after it issue , it remembers the program order and it keeps the results until its safe to write to regs.

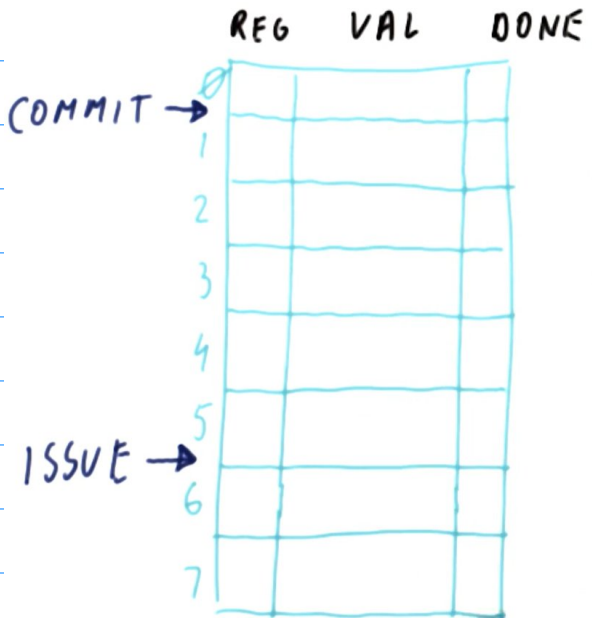
Reorder Buffer
* Remember the program order
* Keep the results until its safe to write

So basically instead of writing to regs whenever results is produced, the results would end up in reorder buffer , and then we will go inorder through the reorder buffer and deposit results into the regs. And only after result is deposited into regs the instn is considered to be fully done.

Now that we know that we need ROB to hold the resluts of the instns until thery are ready to finish in program order, Lets see what it looks like.

ROB-1

The ROB is a table of entries, in each entry we need atleast three fields, The value of produced by instn.(When its broadcasted, instead of writing in the RF, we write here). A bit to tell if its valid. And finally we need to know where we need to move this value (from ROB) into RF, i.e what reg are we going to write it to. And we keep instns in the ROB in the program order. So we need two pointers, one tells us, when we issue the nxt instn. where does it go. another tells us when we finally complete (in program order) the nxt instn , which instn is it going to be. As seen in the fig, there are 5 valid instns in the ROB oldest one being #1, and latest one #5, and nxt instn is going to sit in #6 and so on.

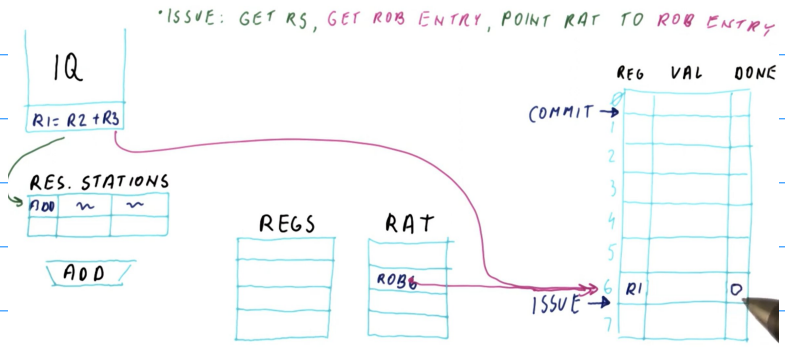


ROB-2

Now that we know the structure of the reorder buffer, we will see how it works. For that we need ROB, and the rest of the proc. that we have seen. We will see what happens once we start issueing, the instn, all the way to once we finally commit this instn. in a proc that now has ROB.

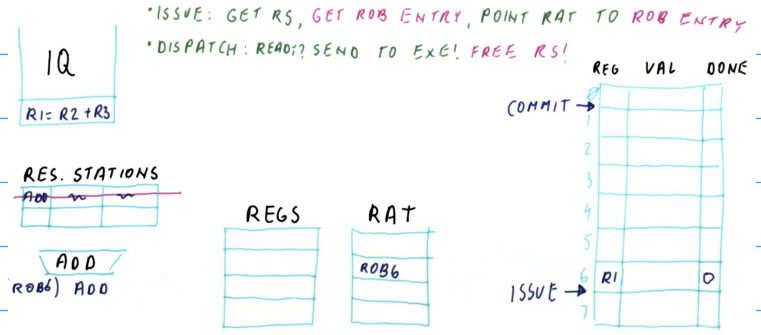
During issue: As before (when we didn't had ROB), We get a free RS, and we put out instn in it. We will also get one of ROB entry.

Unlike RS, in which case we just need a free RS(doesn't matter which one it is,) here we need to get the entry that is nxt to issue pointer. Once we place the instn in the ROB, the issue pointer increments by one (now we will have 7 instns in the ROB, that have been issued and not yet commited ?6?). Another thing that we had to do when we are issueing is,as RAT needed to be pointing to RS (in tomasulo's), now we need to point RAT entry to ROB entry. When we place instn in ROB, we mark it as not done.



Now the add in RS might wait for its operands to become ready, that happens exactly like in the tomasulo's algorithm. Eventually when the add has all the operands, dispatch happens exactly the same way as before, with the change that we can free RS as soon as we dispatch { with tomasulo's , we used to free after the broadcast, that was because the RS serves as the name for the result., now that the name for that result (as can be seen in the RAT) is the rob entry, we no longer need the RS to play that role}. The RS does only what its primary job, which is wait for operands and capture them, and figure out which instns to dispatch, after that the RS for that instn is done, so we can free it.

And now that our ADD executes it will produce result and carries with it, its tag (which is the name for its result),



Free RS Quiz

We have seen that the ROB changes when we free RS, Lets see if we understand that. Suppose that instn can't issue because there is no RS free for it. This is more likely to happen in :

☐ · NO ROB (TOMASULO)
· 2 ADD / 2 MUL RSs

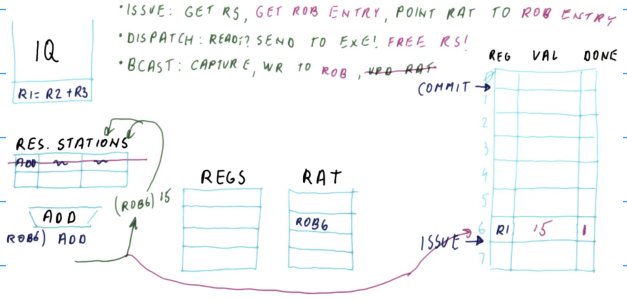
☐ · ROB
· 2 ADD / 2 MUL RS

In case of proc. with no ROB, we hold on to the instns. from the time we issue until the time the instn broadcasts the result. So instns waiting to issue will have to wait for instn.s in RS to broadcast the result. IN case of a proc. with ROB, we grab RS when we issue the instn. and we free them as soon as its dispatched.So we free the RS sooner, thus the nxt instn. might find a free RS , while in case of proc without ROB, it will be still not available. SO the given situation is more likely to happen when we donot have a ROB than when we do.

ROB-3

Eventually we will finish executing instn. and we broadcast the result. It happens exactly as before in tomasulo's, except that the tag we carry for the result is the ROB entry not the RS number. Just like in T, this value and the tag are now given to the RS, and they normally capture the results based on the tag as before. SO who needs it ,captures it. Without the ROB, we would capture this results in RF as pointed by RAT (and invalidate it) . But now,

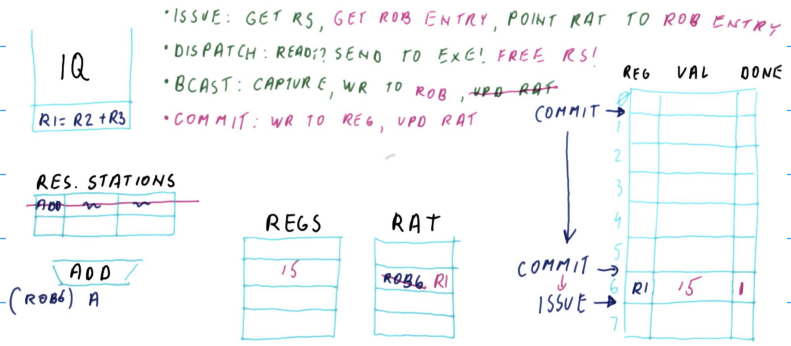
We write to the ROB, and mark the instn as done. Also the result is not going to be in the regs, so we dont update the RAT. In fact the RAT still points correctly, the result of the instn is in the ROB now.



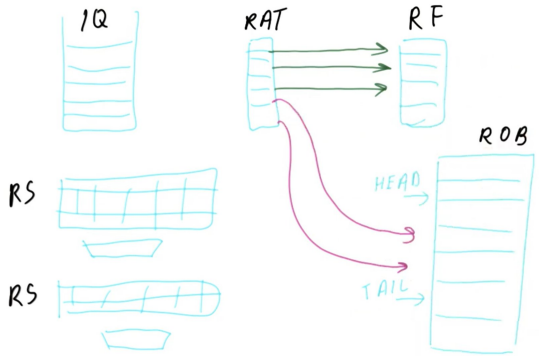
After Bcast, the instn would be completed in the T, but with the ROB, we are still not done with it. Yes we have completed the result and marked the computation as done (in the ROB) , but we have not deposited the results in the regs . For that we have additional step called commit.

TO commit this instn., we must commit all the previous instns. to commit. So in each cycle we check if the instn at commit point is done?, in which case the commit ptr can move on and commit that instn. So eventually the commit point will reach here (to #6), and now we will test whether this instn is done, find out its done and then commit it.The commiting consists of actually taking this value, and writing to the reg pointed by reg field of ROB, and followed by updating the rat (because after the commit, we free the ROB entry) to be invalid. So basically updating the RAT has moved from bcast time to commit time (compared to T), because that's when we are updating the RF. And after commitng this instn the commit pointer will move to point to nxt instn.. And that complets our ROB proc. step by step for one instn.

Many of the things are happening like T. Some things have changed like, pointing the rat to ROB instead of RS. Writing to the ROB on broadcast instead of RF, also we have added a step called commit, also we can free RS on disp., update RAT in new step.



Hardware Organization with ROB



Tail is the commit pointer and head is the issue pointer.

We still have the IQ , from where we dispatch into RS. We still have RAT, that can point to RF, or renamed name for the instn, until it writes to RF (in T that would point to RS). Now we have extra structure called ROB, which has head and tail pointers, the instructions that are currently in execution are between these two pointers([tail, head) inclusion) .Head is ptr to where are we gonna put the nxt instn. that issues, the tail ptr is where we are considering instns for commit. And the RAT entries that are not pointing to the RF, now point to ROB entries that are gonna produce that value.

ROB Quiz

We need ROB to : (select alll that apply)

* We need to remember program order until commit
* We need to temporarily store the instn.'s result until commit
* To serve as name (tag) for the result
* TO store the source operands until dispatch of the instn
* To determine which instn goes to which exe unit

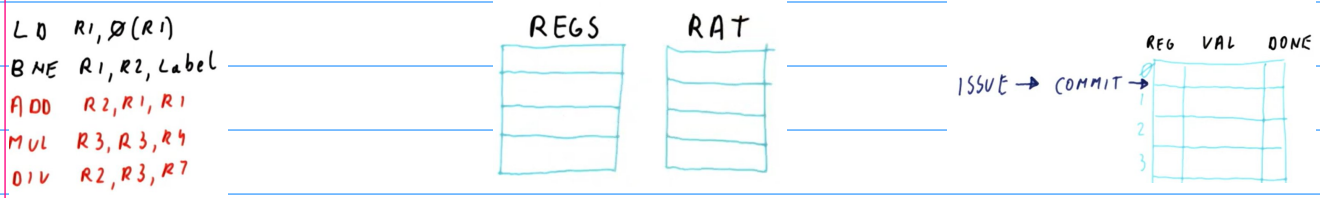
Sol:

* ROB is the only place where prog order is preserved b/w issue and commit and things in between are OoO, so this option is correct
* Yes , ROB holds results b/w time its produced to the time its commited to RF
* Yes
* No , the RS retains this role from the tomasulo's algo
* No , the RS has this role. ROB is usually unified. Diff exe units has difff RS. And when we issue an instn, we issue it to right RS and that determines what is going to be exe unit.

We have seen that we meed ROB to have precise exceptions and to easily be able to recover from branch mispredictions. So let's see how that happens.

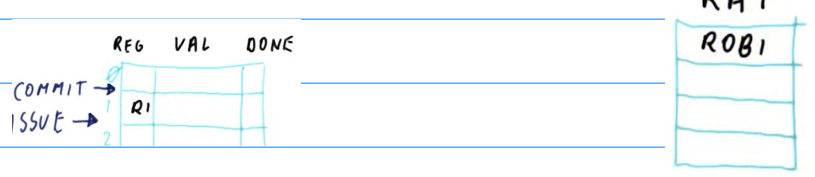
Branch misprediction recovery

Let's say we have (mis)predicted to be Not taken, and see what happens.



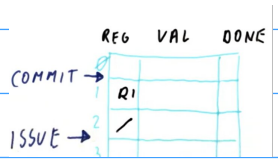
At the beginnig, the issue and the commit pointer are the same, i.e ROB is empty, and no instns are there,so RAT is just pointing to the RF.

Now when we issue the load,

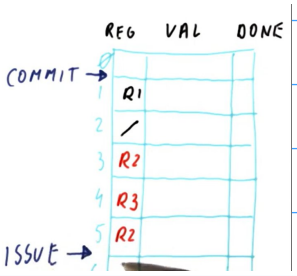


When we issue the branch, it produces no output, so no need to update the RAT.



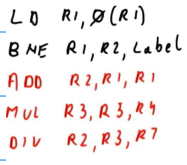


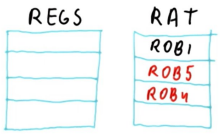
After we issue all the rest of instns. out ROB and RAT look like this. Note that RAT entry 2 will be updated from ROB3 to ROB5, after we issue DIV.

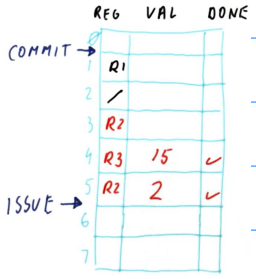




Now suppose that the load takes long time to produce result , because its a cache miss, so the dependent branch instn can't complete until the load gets done, and the ADD also depends on R1, so it will also wait. But the MUL might have completed exe, because its not waiting for any of the results , So once done, it will write results to ROB and mark it as done. Once mul is done, the div starts and eventually it will also be done and value will be written to ROB, this might also be done before the load gets the value. In case of T, we would have written these results to RF, and done permanant damage, but here with ROB based proc., we haven't yet written to regs, just updated ROB entries.

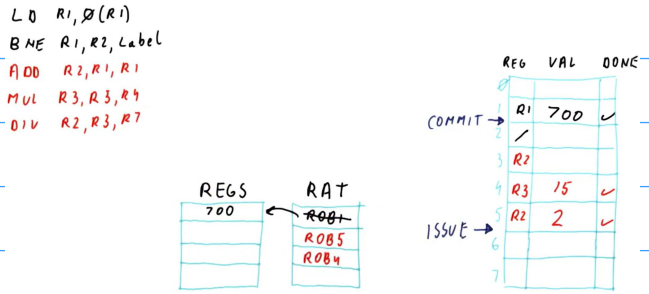






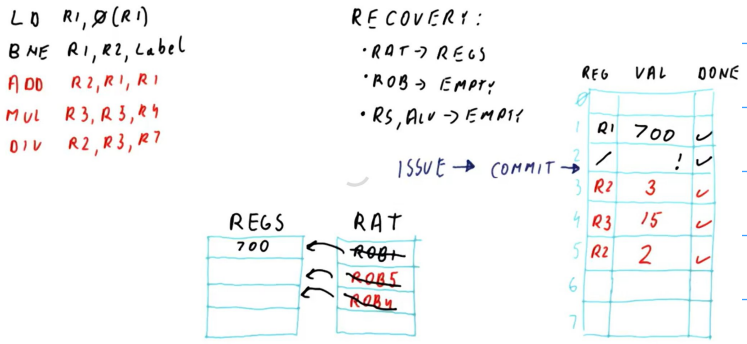
Now, eventually, Load returns and the val is written in ROB and marked as done. Then we will be doing the branch and the ADD that were waiting for load.

Meanwhile we are going to check if we can commit the instn pointed by commit pointer, and find that we can, so we will commit this instn and then move the commit pointer.



Let's say Branch takes longer to figure out than the add, so the add completes and puts value in ROB. Note that this result would have not been written to RF even in T, because RAT for R2 is pointing not to this instn., we would have written the value of 2(from div) because that is the latest R2.

Eventually the branch gets resolved, and we find that its mispredicted. SO now the branch will be marked as done, we will start fetching from the correct instn. addr. But the question is how do we get rid of wrongly exe instns. What we do here is we annotate that there's been a mispred. And when we commit the branch we will figure out that its mispred. And after we commit, We do the recovery before we restart the fetch from the correct place. The steps are we make issue pointer same as commit pointer. (and when we issue new instns we will put them in ROB and unmark the done) and the nxt and last step is clear all entries in RAT. So that each RAT entry correctly points to the RF. Also we will free all the ALU and the RS contents(so in case if any of the instns are in RS or in ALUs that will simply be dropped) . And then we can begin fetching from the correct place.



ROB and Exceptions

Now let's see how our ROB is fixing ROB exception handling problems. THere were two of those problems .

When div gives an exception , (instns after it might have written to ROB), We treat exception as just another result. i.e we write in the ROB that the instn resulted in an exception, not a value. When the instn reaches the commit point, the instns before it have all commited , and instns after it all have not commited. Then we can just flush everything after the div, including the div., jump to exception handler, and now we have a stable state for the handler (which is the from the point of div instn) to , see the state of done instns and to return to when the exception handler returns.

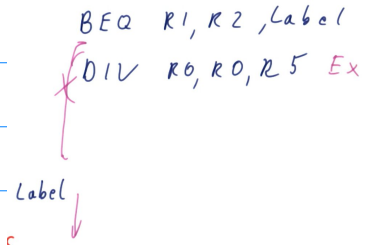
Similarly when a load has page fault, when the page fault reaches commit stage, we have commited everything before it and not commited everyrhing after it (and even page fault itself not commited) . So we have very nice resume point for the page fault exception handler.

The second problem that we had with exceptions were those phantom exceptions, i.e exceptions in mispredicted paths. Say we have div(that is executing due to mispred.) after a branch, and say div triggers divBy0 exception before branch is resolved. So now the result of this (DIV) is marked as Exception in ROB. As the commit reaches that branch, we figure out that the branch has been mispred., and we really want to jump to correct location. At that point , the DIV / anything after Branch will be

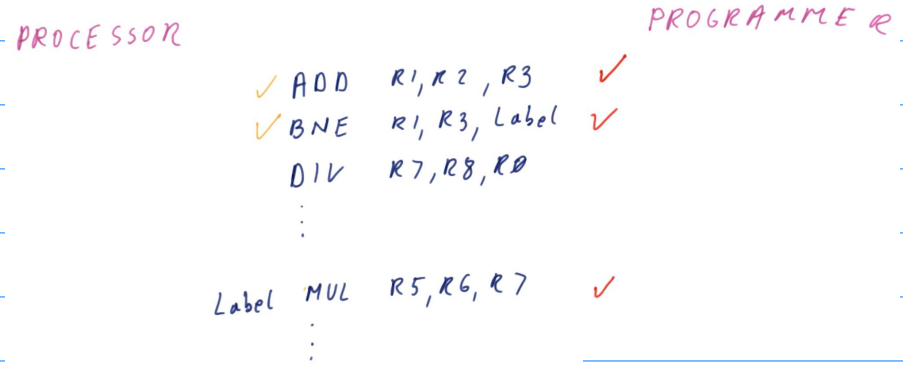
not been commited, and they will be flushed. As Div never reached commit stage, it never triggers exception.

So the idea is

Treat this exception just as any other result, and delay the actual handling of the exception until the instn that triggering the exception commits. At that point we know exactly what the resume point is for the exception handler, and we know we wont have any phantom exceptions



Commit is outside view of executed



Lets see the above example, we do add, we do branch, we mispredict, and we execute div, and later figure out branch was mispred. , then we undone DIV and execute MUL. As far as the proc is concerned, it could look like, we first did BNE, then DIV, then ADD the resolve the branch UNDO DIV and finally continue from MUL (label) . So at some point in time the proc may see DIV and ADD as fully executed (but not commited), but at the same time the programmer sees only ADD as done(because only it is percieved as commited) . SO until the processor finishes BNE, the programmer not gonna see DIV as completed. In fact the programmer never really sees the DIV as completed because ,, BY the time the programmer sees something after

the branch execute, the branch would have been commited, and at that point we flush the mispredicted instns and after that the proc will start fetch from Label, so the nxt thing the programmer sees is MUL. Programmer never sees execution of any wrong path instructions and on exceptions , the prog never sees any instns other than the ones that should have been executed.

So basically the commit is like a officially executed , where as the actual execution in the proc before we commit , is simply kind of just the internal state of the processor that might not be reflected to the outside world, until we are sure that everything is fine with that result.

EXCEPTIONS WITH ROB QUIZ

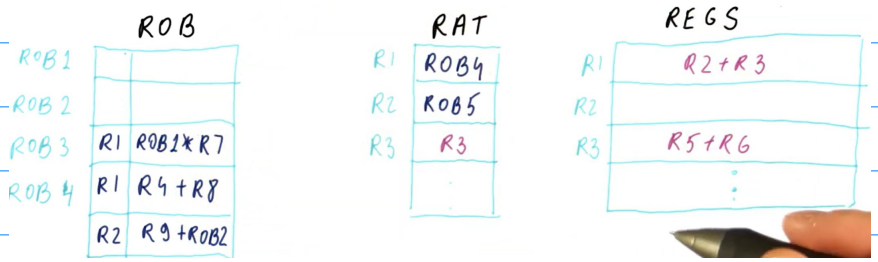
INSTRUCTIONS	STATUS	NEW STATUS	Solution
ADD R2,R2,R1	COMMITTED		Commited
LW R1,0(R2)	EXECUTING		Commited
ADD R3,R4,R5	DONE		Commited
DIV R3,R2,R3	EXECUTING E		Undone
ADD R1,R4,R4	DONE		Undone
ADD R3,R2,R2	DONE		Undone

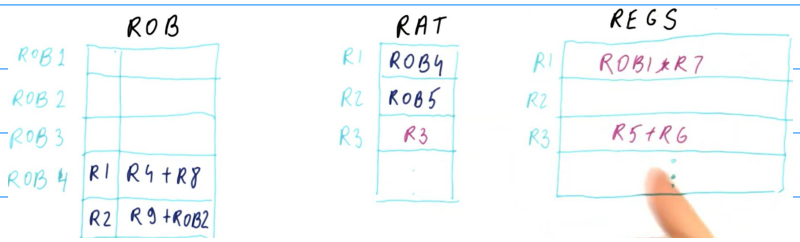


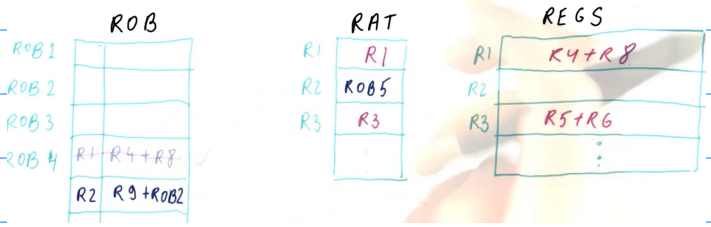
Undone or Unexecuted meaning they behave like that instns haven't been fetched. When commit sees the exception, it undo s all the instns after it including the instn that gave exception and jumps to exception handler , it acts very much like a branch misprediction except that it happens on an exception.

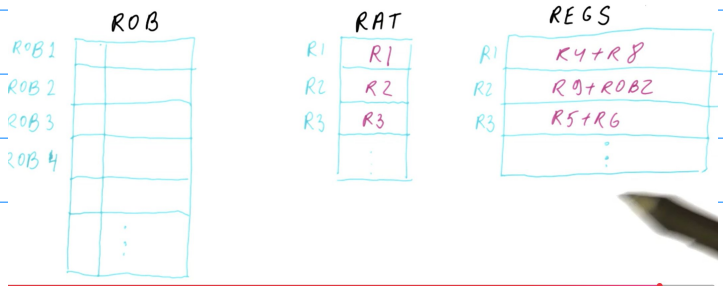
RAT Updates on Commit

Each time we commit the instn, we deposit the values in the RF, regardless of whether this is the latest value for that register (unlike we do in the T by seeing the RAT)

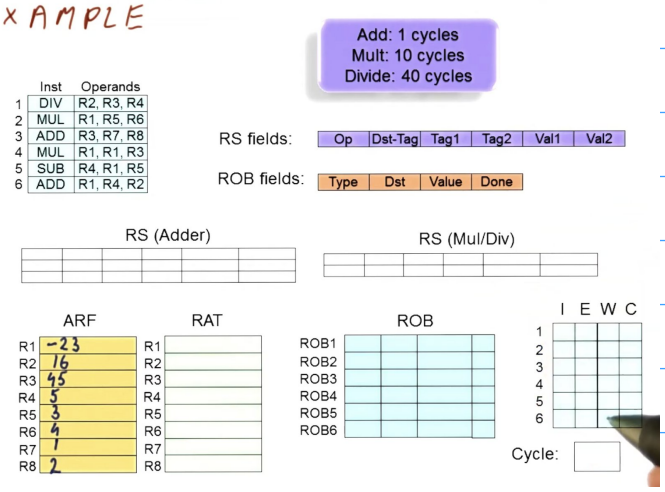








ROB EXAMPLE



In cycle 1, we try to issue the DIV, (to do this we need an available RS

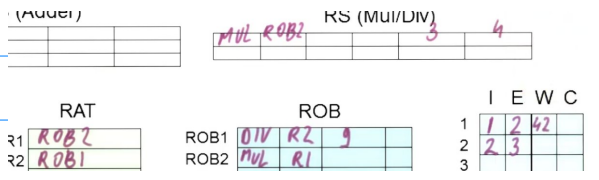
and a ROB entry, we have both so), we issue,

* put DIV in ROB1 and in RS
* put dest. tag as ROB1
* see rat to check where we can find out operands, if in RF, get values
* rename R2 in RAT to point to ROB1
* Because we said that we can fetch and dispatch in same cycle, we disp
* as we disp in this cycle, exe going to start in cycle 2



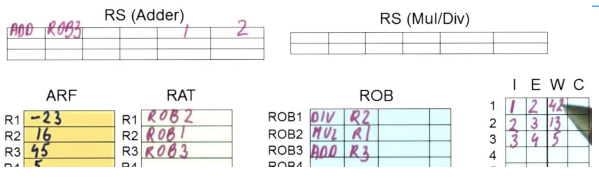
In cycle 2:

* DIV begins executing so RS becomes free, we can note the result in ROB,
  later when it completes, we can mark it there, we can also note
  it writes at cycle 42 in the cycles table.
* We can issue MUL , so we take ROB and RS entry, we see rat and get their
  values, we can dispatch in same cycle, we will rename R1 in RAT,



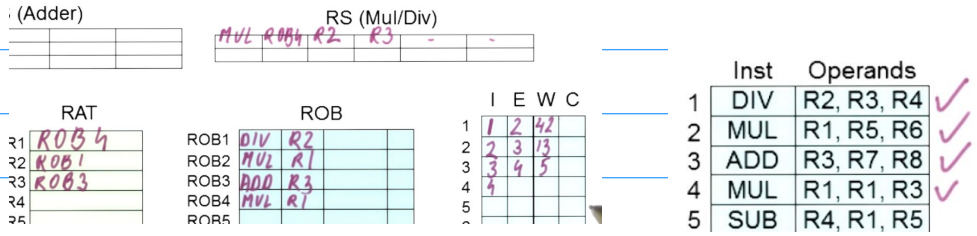
In cycle 3:

* Mul begins exe, so RS becomes free, we try to issue ADD , we have ROB
  and RS, we see rat and get the values, we can dispatch in same cycle,
  we will rename R3 in RAT to point to ROB3, we can start exe in nxt cycle,
  results will be written in nxt2nxt cycle.



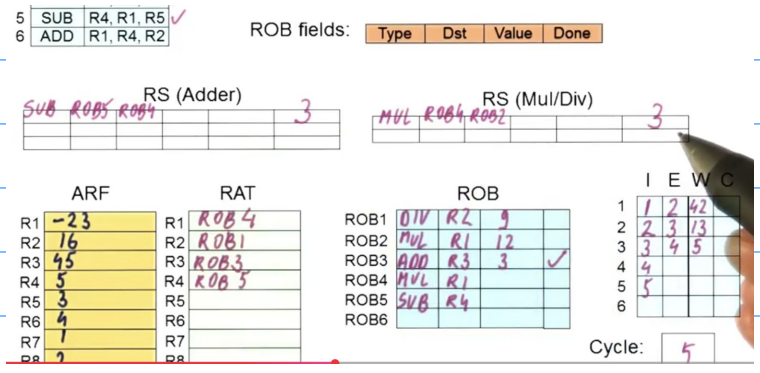
In cycle 4:

* At the begining we have 3 free RS(becaused we have dispatched in prev cycle) .
* As we have free RS and ROB, we issue MUL, we write its a mul in RS and ROB, we look at RAT and find R1 and R3 can be found at ROB 2 & 3 respectively, so we put them in RS tag fields, then we rename R1 to be ROB4 in the RAT(be careful you rename RAT after you read your operands , otherwise instn can end up waiting for its own result, deadlock) ,
* This instn will not be able to dispatch
* Add is executing
* And nobody is broadcasting



In cycle 5:
The add is trying to write its result, and we can try to issue another instn, we can do in either order.

* We issue SUB, we put in ROB and RS, we read the locations of operands in RAT, and find we can't dispatch, so we wait in RS, then we rename RAT
* ADD writes result with the tag as ROB3 and the instn is marked as done, RS(Mul/Div) captures it
* Neither of the instns are ready to dispatch in cycle5



In cycle 6, we try to dispatch instn 6, as far as exe is concerned, both the waiting instns can't dispatch as their operands aren't ready, as far as broadcast is concerned, we have add finished broadcasting.

* So we only issue ADD, find a place in ROB and RS, read operand locations from RAT, and update R1 entry (to point to ROB6) in RAT.

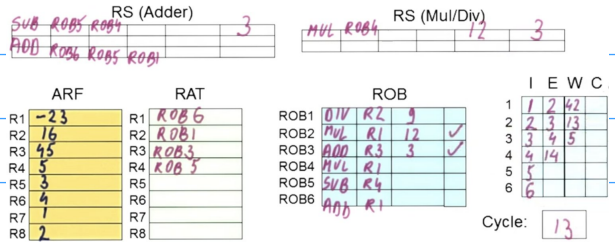
In cycle 7:

* we can't issue anything as we have issued all the instns. (also no ROB entry)
* execute, we can't dispatch anything as we are waiting for operands to become ready

So nxt cycle something instresting happens is cycle 13 when mul broadcasts

In cycle 13:

* Mul broadcasts 12(ROB2) and marks as done in ROB
* Mul in RS matches the tag, so it captures and dispatches, so exe in C14

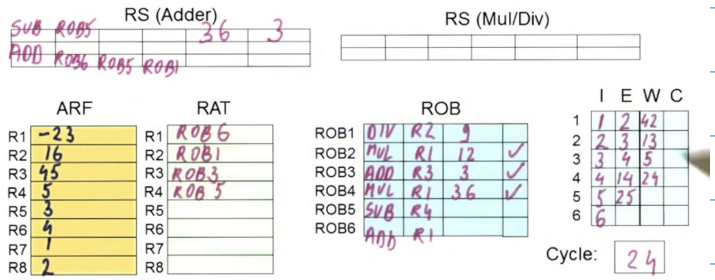


In cycle 14:

* Mul begins execution. because this instn dispatched in previous cycle, this RS is already free. So we can remove it. We may note its value in ROB4 and will mark it as done when it completes execution (which is cycle 24, we can mark it in cycles table)
* We can't execute any other instn as rest instns are waiting for operands
* Nothing really happens until we broadcast

Cycle 24:

* The value of 36 (with tag ROB4) will be broadcasted, marked done @ ROB4
* Sub will capture it and will dispatch, it will produce 33 at cycle 25 and will broadcast in cycle 26



In cycle 25:

* Sub will start exe , and will produce result in 26, RS of this SUB will be free at the start of the cycle.

In cycle 26:

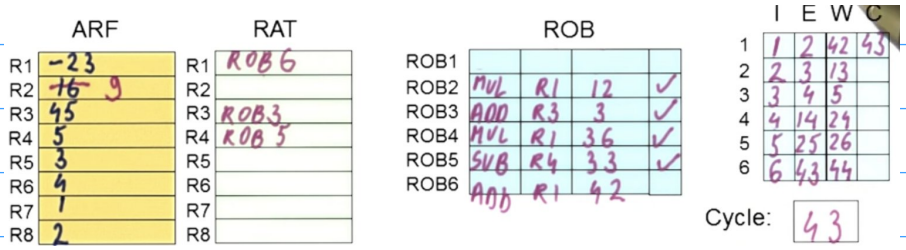
* Sub broadcasts value 33, and will mark as done in ROB5,
* Instns waiting in RS, the ADD will capture the value

In cycle 42:

* 9(ROB1) will be broadcasted and marked as done in ROB1, ADD in RS will capture and dispatch, so ADD begins exe in cycle 43 and broadcast in 44

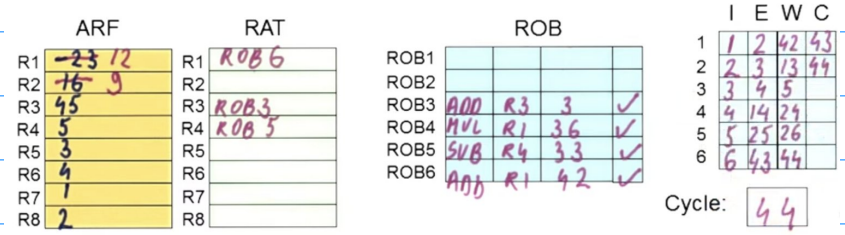
In cycle 43:

* At the start we can find its RS free and it will start exe .
* DIV can commit (as its marked done in ROB)
    * Update RF by seeing RAT entry
    * Invalidate RAT if its pointing to our ROB entry
    * Free ROB entry



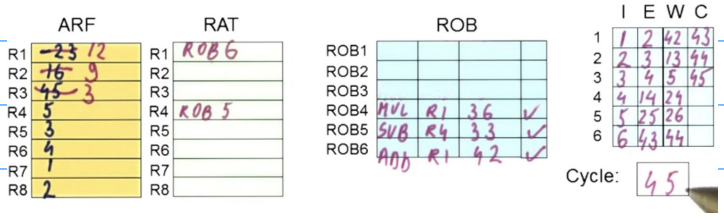
In cycle 44:

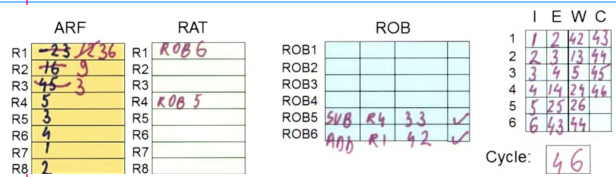
*We will be broadcasting Instn6 , we will mark it as done in ROB
*Commit MUL
    * Update RF by seeing RAT entry, so here we don't update
    * Invalidate RAT entry if its pointing to our ROB entry, no
    * Free ROB entry, yes

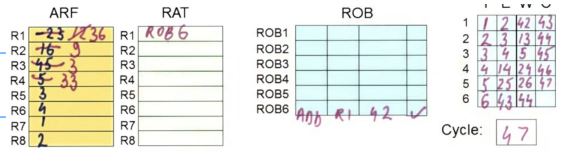


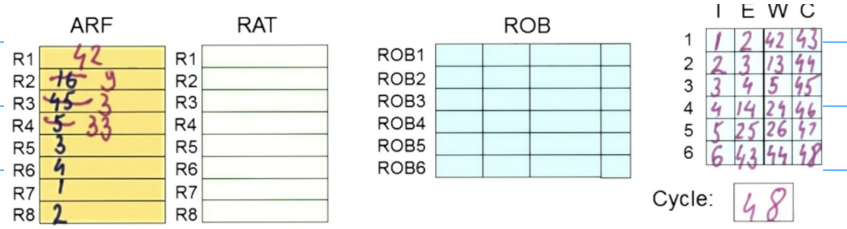
In cycle 45:

*Commit ADD

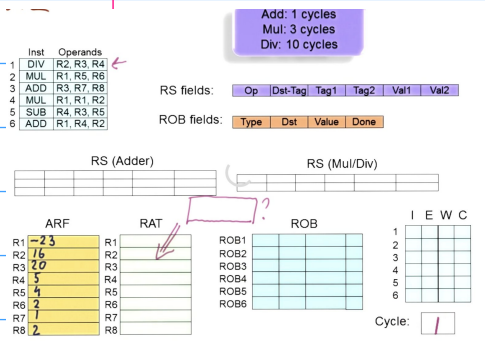








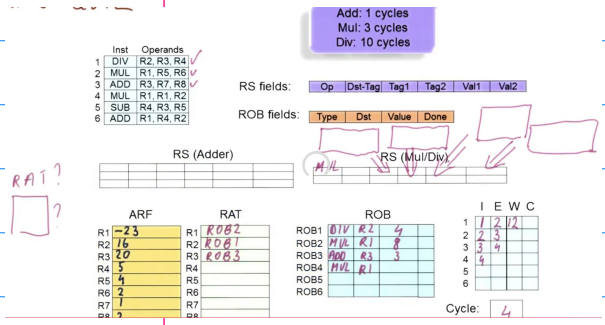
ROB Quiz



Assumptions:
Same as before example
issue-dispatch same cycle can happen
broadcast starts nxt cycle after exe is done

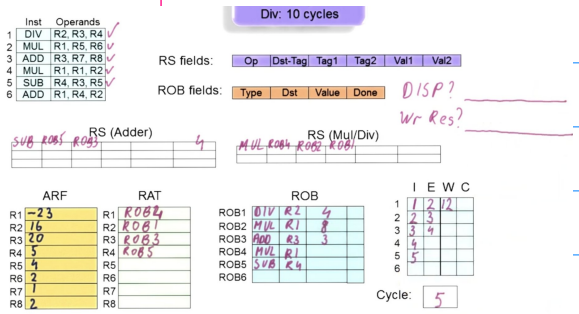
We try to issue DIV, What is the value of R2 in RAT

Sol: There is space in RS and ROB so, div gets in RS and in ROB, gets its values and dispatches, So R2 will get ROB1 in RAT.



We try to issue MUL in cycle 4, what values/tags go in RS, and which RAT entry is modified

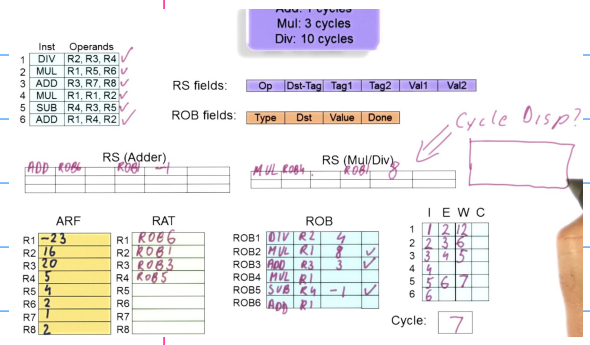
Sol: Tag1: ROB2 , Tag2: ROB1 & Dst tag:ROB4
RAT's R1: ROB4



Which instn writes result, and which instn disp, if any, write the instns name as ROB entries

ROB3 instn writes result, so we can mark it done in ROB3, So sub waiting for it will dispatch and will begin exe in cycle 6.

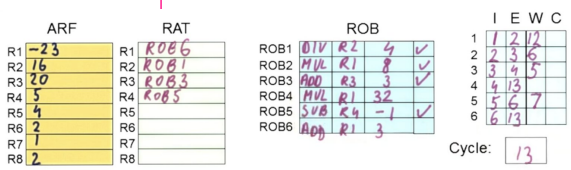
Note that, SUB issued , captured result and Disp in same cycle i.e cycle 5



In which cycle does MUL dispatch,

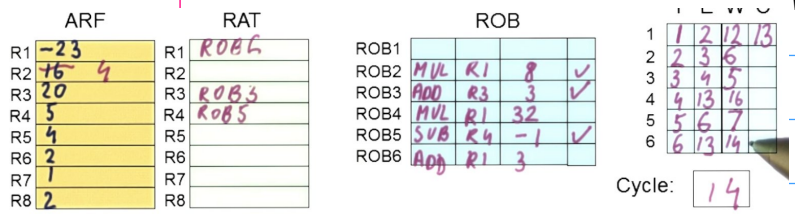
Sol: It will dispatch in cycle 12

Apart from exe, what happens in cycle 13



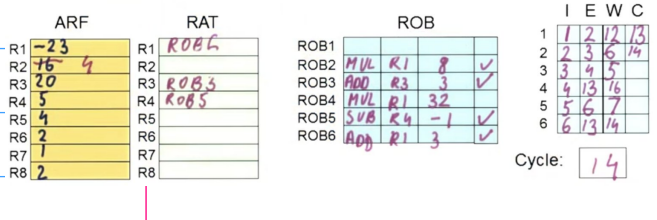
Commit of ROB1 instn , so steps are:
* We write to RF the value 4 in R2
* Invalidate RAT's R2 and
* ROB

We can't write results of both executing instns



Which RAT entry changes

As a result of writing the instn6 result, nothing changes in the RAT, because Broadcast doesn't update the RAT, commit does

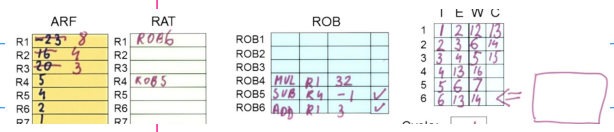


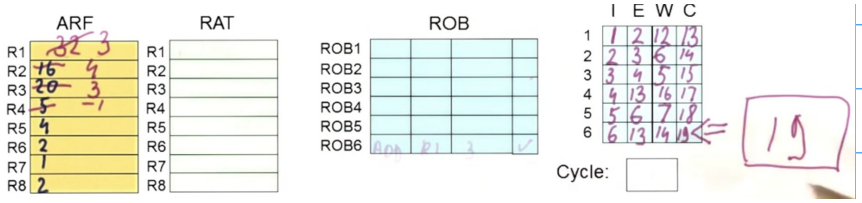
Which Architectural Reg entry changes, on commit of ROB2, if anything changes to what value it becomes.

Sol:R1 changes to 8
We just free the ROB entry , no changes to RAT.

When does the last instn commit
Sol:`

19





ROB Timing example

- FREE RS ON BCAST, NOT DISP
- ISSUE, CAPTURE, DISPATCH IN SAME CYCLE

Add: 1 cycles
Mult: 10 cycles
Divide: 40 cycles

Say you have 3 add/sub RS and 2 mul /div RS.

Inst	Operands	Is	Exec	Wr	Commit	Comments
DIV	R2, R3, R4	1	2	42	43	
MUL	R1, R5, R6	2	3	13	44	
ADD	R3, R7, R8	3	4	5	45	
MUL	R1, R1, R3	14	15	25	46	NEED RS → ISSUE
SUB	R4, R1, R5	15	26	27	47	EXE DEP ON R1
ADD	R1, R4, R2	16	43	44	48	EXE DEP ON R2

ROB Timing Quiz

- BCAST ONE ADD/SUB AND ONE MUL/DIV PER CYCLE
- COMMIT UP TO 2 INSTS/CYCLE
- FREE RS AT DISPATCH

Add: 1 cycles
Mul: 2 cycles
Div: 4 cycles

IS: _____ EX: _____ WR: _____ COMMIT: _____

Inst	Operands	Is	Exec	Wr	Commit	Comments
DIV	R2, R3, R4	1	2	6	7	
MUL	R1, R5, R6					
ADD	R3, R7, R8					
MUL	R1, R1, R2					
SUB	R4, R2, R5					
ADD	R1, R4, R3					

Issue happens in C2, Exe in C3,

WR in C5 & Commit in C7:N C8:Y

Because we still have 2 mul RS, & 3 add/sub RS.

We can't commit two instns of same exe unit in same cycle..?

Quiz 2: When does the nxt 2 instns issue and commit

Sol: ADD: 3I 4E 5W 8C (cmnts: we have 2 broadcast buses)
Mul: 4I 7E 9W 10C

Inst	Operands	Is	Exec	Wr	Commit
DIV	R2, R3, R4	1	2	6	7
MUL	R1, R5, R6	2	3	5	8
ADD	R3, R7, R8	3	4	5	8
MUL	R1, R1, R2	4	7	9	10
SUB	R4, R2, R5				
ADD	R1, R4, R3				

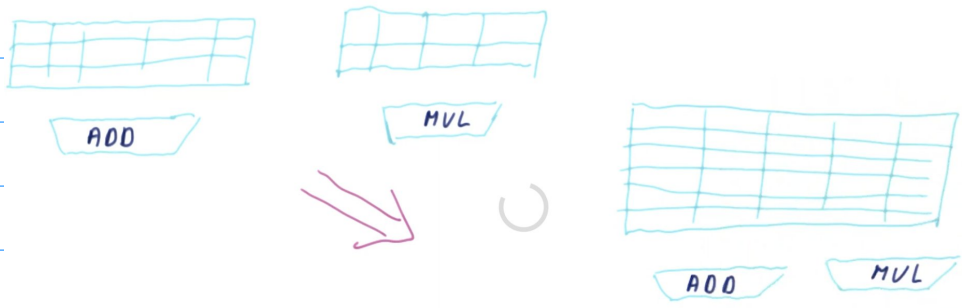
Quiz: When does the last instn commit ?

Sol: Sub: 5I 7E 8W 10C
ADD: 6I 9E 10E 11C

We can start E of Sub in 7, because we have exe unit available, if the previous instn were add/sub, then we could have not be able to do

Unified Reservation stations

The structure of ADD and MUL RS is exactly the same. Say we can't issue a mul because its RS is full but say ADD RS is empty. SO to improve utilization of these expensive RS, (as the functionality of both of them is same, wait , capture and dispatch), we unify them. The drawback is that , the logic to dispatch these instns into respective RS becomes a little complicated. SO in regular RSs if more than one instn is ready, we used to select only one to dispatch based on some heuristics. But here we need to select one instn for ADD EU and one for MUL exe unit. So we need to consider, which one of these are adds and which one of these are MULs to go to their EUs. But the advantages outweigh because RSs are very expensive, so processors usually use unified RSs



Superscalar

- FETCH >1 INSTRUCTION/CYCLE
- DECODE >1 INSTS/CYCLE
- ISSUE >1 INST/CYCLE
- DISPATCH >1 INST/CYCLE
- BROADCAST >1 RESULT/CYCLE
- COMMIT >1 INST/CYCLE

"WEAKEST LINK"

We have seen a proc that can commit two instns/C, but that was issueing ony 1 instn/C. Now lets see what a true

Superscalar processor looks like:

*The Dispatch, to have multiple disp on avg, we need to have greater # of ADD EUs than MUL/DIV. *Also Cost of RS is directly proportional to Broadcast buses, because we need to monitor those many Busses.

*Commit and issue are inorder, and if multiple are ready, we do first one and see if subsequent ones are ready.

* We want all of them to support that instn throughput , its like pipeline and the narrowest one limits our overall IPC.

Terminology Confusion

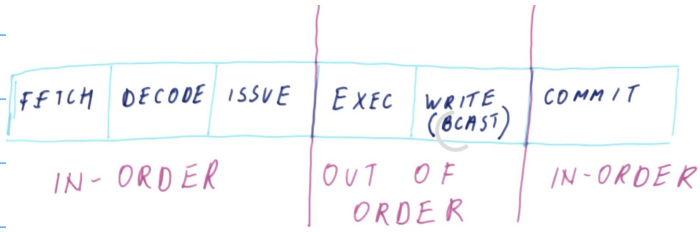
- ISSUE
  - ISSUE
  - ALLOCATE
  - DISPATCH

DISPATCH
- EXECUTE
- ISSUE
- DISPATCH

COMMIT
- COMMIT
- COMPLETE
- RETIRE
- GRADUATE

In research literature, these terms are used interchangably.

Out of Order Things in ROB based Processor



OoO Processor Quiz

STAGE ↓	IN-ORDER	OUT-OF-O.
FETCH		
DECODE		
ISSUE		
DISPATCH		
EXECUTE 1		
EXECUTE 2		
BROADCAST		
COMMIT		
RELEASE ROB ENTRY		

Mark which stages are inOrder and which ones are OoO

Only Disp, Exe1, Exe2 & Wr are

Memory Ordering Intro

We have seen that an OoO processor will track when an instn uses a reg produced by another, but load and store can access the same mem location without using the same reg for it. Lets see how to do them correctly in OoO processor.

We have seen by using ROB and tomasulo's we can enforce the order of dependencies on regs between instns, the question still remains what about mem access order, Ld/St instns do they need to be done strictly in program order? Or can we do something about reordering them too.

Note that the data dependency handling we have done so far is only for reg dependencies. i.e when one instn is dependent on reg that is produced by another instn.

But what about mem dependencies, if we have a Ld after a St, there could also be a dependence between the mem loc written by the St and the one which is read by the load. If these two addr. are the same, then the Ld needs to get the value from the St., in which case we should be doin them in order, however, if the addr. are not the same, just like in the case of regs, we don't need to maintain the program order, between these accesses.

So lets see something that can handle OoO execution of mem instns.

When does the Mem Wr Happen

It happens at commit, because, its unsafe to wr to mem at any point before commit, any instn that is not yet commited is subject to cancellation(for example because of a br mispred or an exception). So doing mem access prematurely means if we were wrong we need to undo the access(i.e we need to put back the old value at that addr. which is extremely difficult to do). So better we delay wr until we commit.

But does it mean that the load has to also wait until commits before it can get the data ? (because only then the data is in the mem)

For this purpose we introduce LSQ (Load store Queue). Thats where we keep all loads and stores , and lets see how we can get load data from stores.

As we have seen loads needs to be done as soon as possible, while the stores do not write to mem until commit, we said we need a load / store queue, so that we can supply the value from store to load.

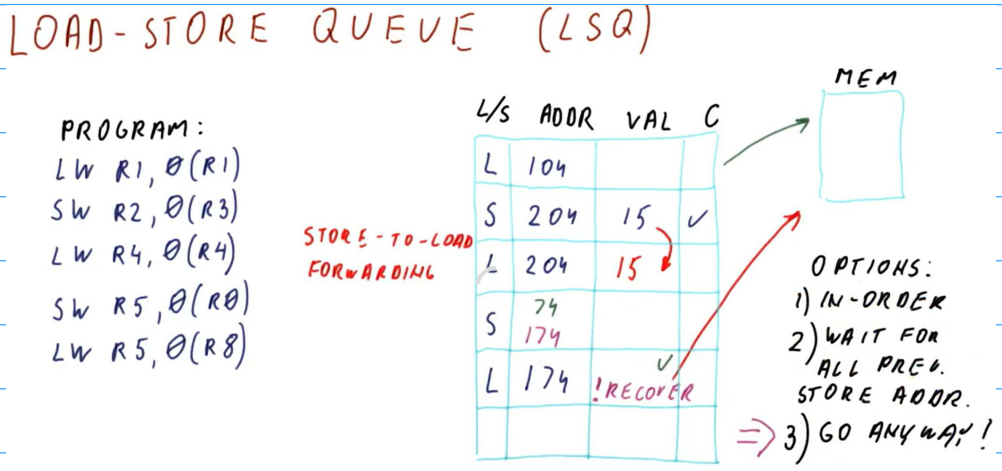
This LSQ is a structure just like the ROB, meaning we put the values in order & remove them at commit, but we only put ls/st instns there. So there's a bit that says which instn do we have (L or S), there is a field that says which location its acessing , the value , then the complete bit.

Lets see an example

* First is load, say we quickly computed addr to be 104, this load will goto memory, because there are no prev stores.
* Nxt instn is st, say its addr is 204, and value is 15, and we mark it done, say we haven't yet got the value from the prev load
* Say this load computes its addr. to be also 204, and now we check prev Stores in LSQ if any has same addr, and the latest one we pick and forward its value to this instn. we call this st to ld forwarding, in which case we never goto mem.
  This , we assume that at the time, (we need value for ld,) st is completed.
* Say we issue nxt st and follwed by load. Say the addr computation for the st is delayed and we sent req to mem for the later (ld) instn
  The options for this situation are:
    * execute ld and st in order, so we can't do this load until all prev instns are completed. So it guarentees when we need value, all prev st are done so we can check for forwarding. But it is very slow if ld is cache miss, then it takes too long to complete and all follwing instns will be waiting for it.
    * The second option is we dont wait for all instns but we will wait for all the prev store's addr to be known. So we can only fetch data from mem if forwarding can't be done.
    * Fetch anyway, if no prev st (that has addr generated) can forward us value, then go to mem. We ignore the fact that some of the st addr can match once we resolve the addresses, and go mem anyway.\

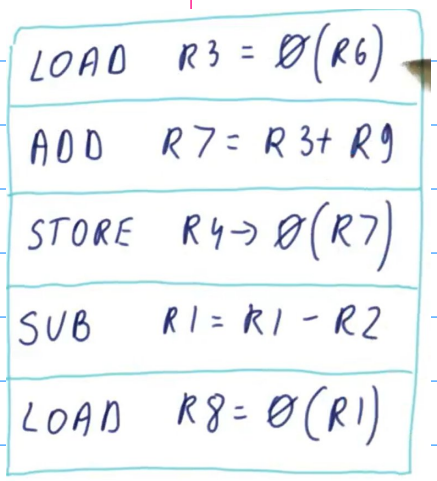
* In go anyway option, two things can happen, the previous St resolve, and we find that the addr. is not the same as ours. OR the st might get the same addr as ours, in which case what we fetch will be wrong, in which case we need to recover, this load loaded a wrong value and possibly supplied it to subsequent instns.
So when a store produces addr., it checks all following loads if any of them matches the produced addr., if matches, it requests redoing of all the following instns including the load, to fix the wrong value (loaded by load) problem.

Most modern processors do this approach, because it turns out this is fastest overall. (Also there are entire schemes in some proc. that try to predict when is this load going to depend on prev store. and on loads that often conflict with prev stores, wait for prev stores to complete.)



OoO Load store execution

Lets see an example of the "Go Anyway" approach.

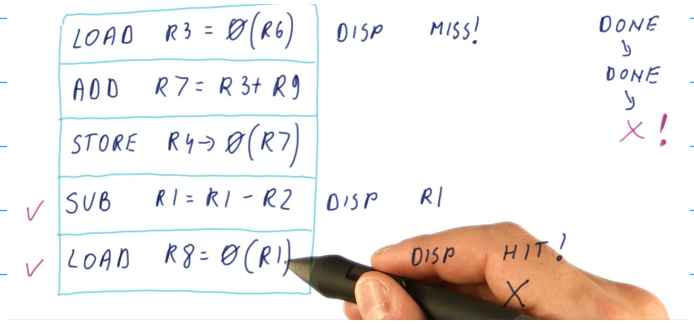


Lets say we have fetched, decoded all these instns., in an OoO processor, this is what might happen:

* Load as not dependent on any prev. instns, will dispatch, meaning it goes to mem (eventually comes back with R3), say its a cache miss, and takes long to come back. ADD and Store being dependent of load, can't dispatch.

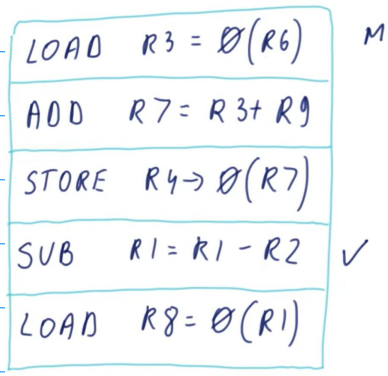
*Meanwhile, Sub can dispatch, and produces R1 very quickly. At this point the last load can disp, say its a cache hit, and maybe supplies R8 value to subsequent intns. . At this point SUB are Load are completed. We just waiting to commit them.

*Eventually load returns R3 value at somepoint, once its done, ADD gets done very soon after. Now we feed R7 to the store. Say if this R7 is not equal to R1, then everything is gonna be fine. Because both addresses are diff. The problem occurs when the addr. the store resolves to is same as addr. that subsequent load loaded from, in which case load should have got R4, but it got stale val.



In Order Load Store Execution

Our first solution to this is , we don't do OoO , but do In-Order Ld/St execution.



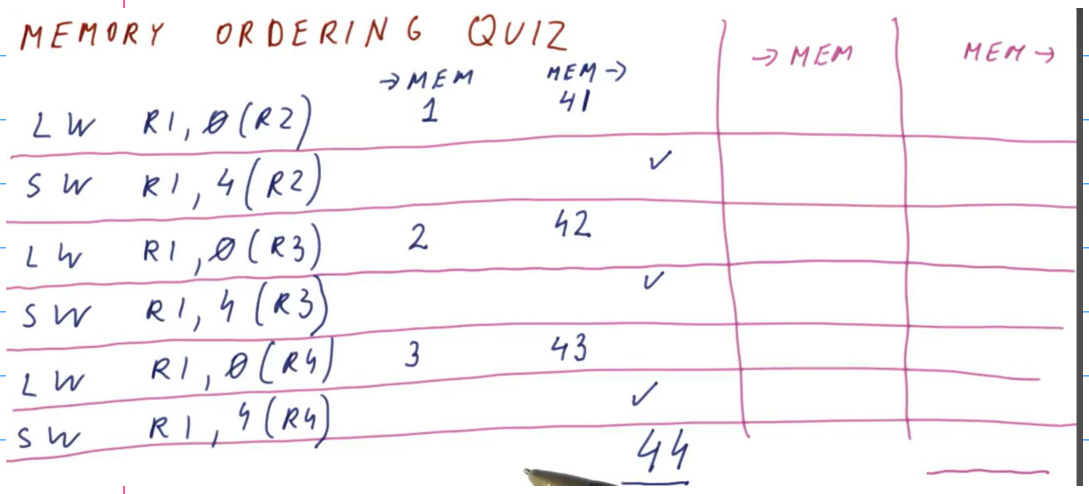
If we were doing inOrder for Ld/St, then we would not dispatch store and following load even if their operands are ready (for last load) until the first load completes. The order of exe is , when load1 gets done, sub is already done, then ad gets done as its dependent on load1, then store gets done, (store is considered done, not when it commits, but when its addr. gets computed (also its value)) only then the nxt ld/st instn is allowed to disp. , so at this point the load2 can go up and check if any stores produc the value for the addr we are looking to load from, if so we can grab the value from there.

So basically we are executing most of the instns in OoO but ld/st are proceeding in Order. Ofcourse this is suboptimal if the store doesn't store to the same addr, from where subsequent load is goin to load from, in which case we have wasted time waiting for store, all the dependent instns are also delayed, this would be even worse if the load is a cache miss, in which case we would have to wait again.

So this is not a very high performance solution, but its a correct solution.

Mem Ordering Quiz

Say all of the loads are cache misses, So say if we disp load in this cycle, say we get value 40 cycles later. Say we are doing OoO exe of ld/st, then in C42, St1 will be done, in C43, St2 will be done, in C44, St3 will be done. Here we have been doing loads prematurely as far as stores are concerned (so we decided to be "go anyway" approach, and we are assuming there will be some recovery mechanism) The task is when is each of these instns sent to mem and when do we get results from mem. And whats the final cycle count, if we do things in Order.



The store that depends on load, will wait to execute until prev load to complete, so first store completes in C42. Then the following ld starts. As it can be seen it took almost three times as long in this case to do the same compared to OoO. So there is a huge advantage , trying to reorder load store instns, but it carries a risk to recover from, basically loading the wrong value from mem (in case we should have loaded the value from the store).

Before we discuss how the recovery from loads being done tooearly is done lets first see store to load forwarding problem.

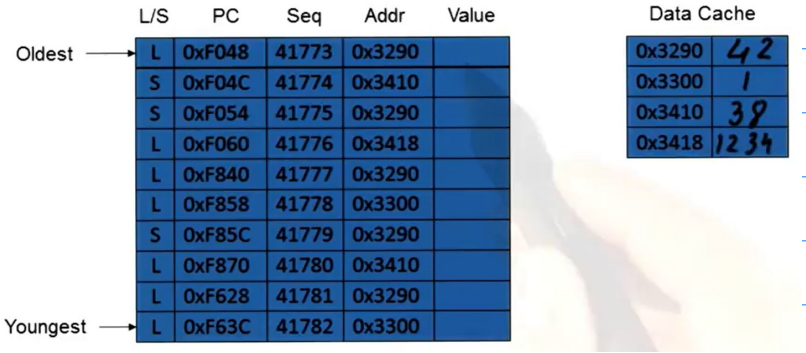
Store to Load Forwarding

FOr loads, which earlier store do we need to get our value from,
So there could be multiple stores previously that our load is going to, and we have to figureout which store supplies that value , and if none are matching (we need to know that) Then we need to goto memory

For Stores, Which later load do i need to give my value to, So there might be a load whose addr is already known, once the store knows its addr and value, we should be waking up the load that wants this value.

We do these things in the LSQ

LSQ Example



First L gets 42
1st St say computes the value it stores to be 25
2nd St need to store say -17, we don't yet send the values to mem because that happens when we commit.
2nd L gets 1234
3rd load searches the stores above it to find if any has same addr it finds a st(2nd) gets value from there i.e -17 (not 42)
4th load gets 1 from cache
Say the 3rd store gets value to be 0, to be stored in mem
The last three loads get 25, 0& 1 (Why?)

The values now are 42, 25, -17, 1234, -17, 1, 0, 25, 0, 1

Now at some point the first load commits(just copy that value to RF)\
Now the commit pointer moves ahead to point to 1st St

The store overrides Cache with 28, and it commits

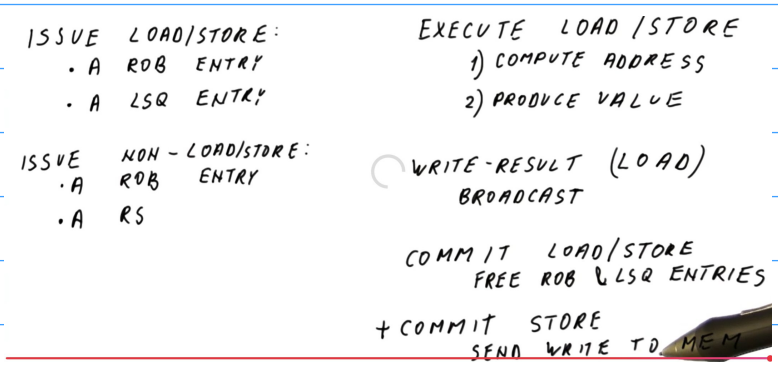
We commit another store , we write -17 in mem

Say at this point we get an exception , we can flush all the instns below the Commit pointer in the LSQ, and the value in the Data cache remains intact. And the final value is 0, 1, 25, 1234

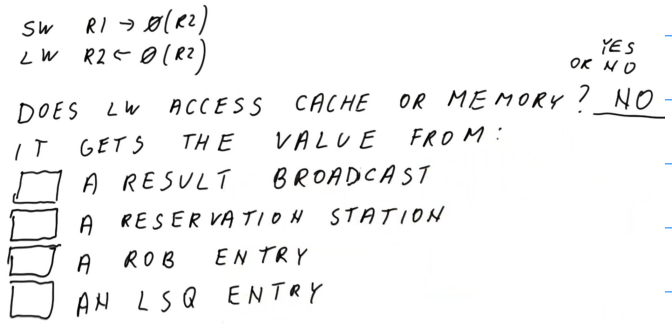
LSQ, ROB & RS

We can't issue a non Ld/St instn unless we have a RS entry & ROB entry,

Similarly we can't issue Ld/St instn unless we have a ROB & LSQ entry.



Memory Ordering Quiz



Select all the options where a Load can get a value from.

In this situation we get from Store (store keeps its value in LSQ),

Store never broadcasts, (it captures broadcast), so not from result broadcast

RS captures values not produce them, also Stores doesn't have RS

ROB holds value of a (register value producing instn's) result between the time its broadcast to the time it commits, but as St is not a register value producing instn., Ld can't capture value from ROB

Load searches value trying to match its addr with prev store's addresses

In this lesson we have learnt we need LSQ to track dependencies through memory, so we can ensure correct execution of mem instns in an OoO proc. in an modern processor, we have really fancy ld/st queues which helps them do a lot of reordering even for mem instns.

We have completed discussing OoO processors that try to do more than 1 instn per cycle, now we will see how the compiler can help us execute more than one instn per cycle.

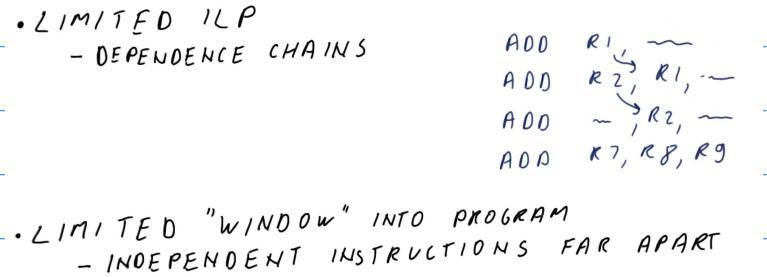
Compiler ILP

Can compilers help improve ILP?

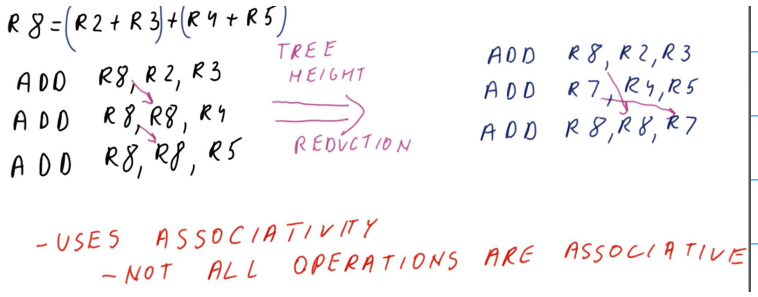
There are really two things that the compiler can help with:

* The compiler might help us avoid long dependence chains

* If there were long dependence chains , there can be independent instns far apart in the program that the proc can't see (can be due to very limited RS/ ROB size. As we will see , the compiler can help us put these independent instns closer, so that the IPC achieved by the processor is closer to the ILP



Tree Height Reduction



Quiz

r1+r2-r3+r4-r5+r6-r7 is the eq.

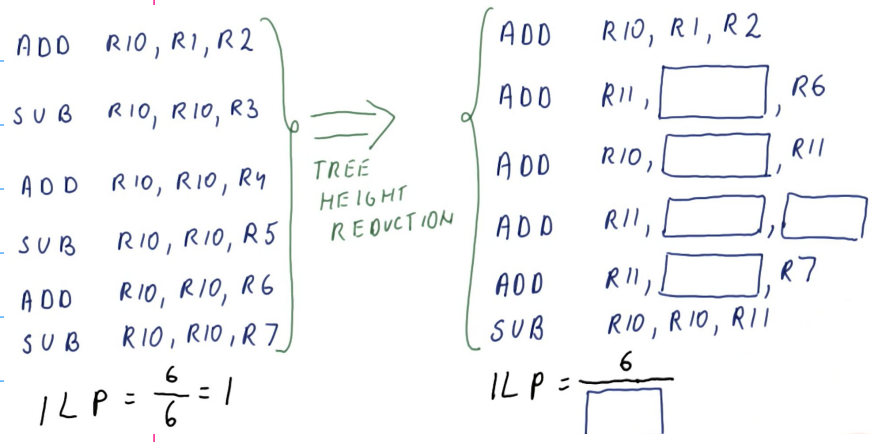
r1+r2 + r4+r6 -(r7+r5 + r3)

r4

r10

r5, r3

r11



1st, 2nd, 4th, can go 1st cycle

3rd 5th can go 2nd cycle

last can go last cycle

ILP is 2.

See predication before continuing with later lectures

Make independent instns easier to find for a processor

We will look at techniques that make independent instns easier to find for a proc. An ideal proc would find dependent instns because it is able to look at inf no. of instructions potentially , but a real proc. can only see some finite instns. so it can't see there are indep instns. further down the code.

*Instructionk scheduling : For simple branch free sequences of instns.

* Loop Unrooling: its a technique specific to loops

We will also see how loop unrolling interacts with instn scheduling

Then we will look at more powerful technique called

*Trace scheduling

Instn Scheduling

Note that this is diff from what would happen in a tomasulo's type of algo (where the HW is trying to reorder instns in a diff way than they were arranged by the compiler). Now here a similar thing is done by the compiler.

The given code is some loop that adds R0 value to all elements of an array that is stored at addr. pointed by R1, and has R3 no. of elements.

LOOP:	**LW R2, 0(R1)**	LOOP:	LW R2, 0(R1)	LOOP:	LW R2, 0(R1)
			STALL		ADDI R1, R1, 4
	ADD R2, R2, R0		ADD R2, R2, R0		ADD R2, R2, R0
			STALL		STALL
	SW R2, 0(R1)		STALL		STALL
			SW R2, 0(R1)		SW R2, -4(R1)
	ADDI R1, R1, 4		ADDI R1, R1, 4		
			STALL		
	BNE R1, R3, LOOP		STALL		
			BNE R1, R3, LOOP		BNE R1, R3, LOOP

Lets say we have a very simple processorit can only look into very next instn it doesn't have RS, and can't exe OoO So the program goes like:

Say LW takes 2C to load from cache & Say ADD takes 3C

So overall it takes 10C ,and we want to help this proc using compiler opt.
In above example we have eliminated 3 stalls with instn reordering.

**

Instn Scheduling Quiz

Say this is the sequence of instns that we have to schedule, ADD takes 1C, SW takes 1C and LW takes 2C. So on a In-Order 1IPC processor, how many cycles does this program take with and without scheduling.

```
LW  R1, 0(R2)
ADD R1, R1, R3
SW  R1, 0(R2)
LW  R1, 0(R4)
ADD R1, R1, R5
SW  R1, 0(R4)
```

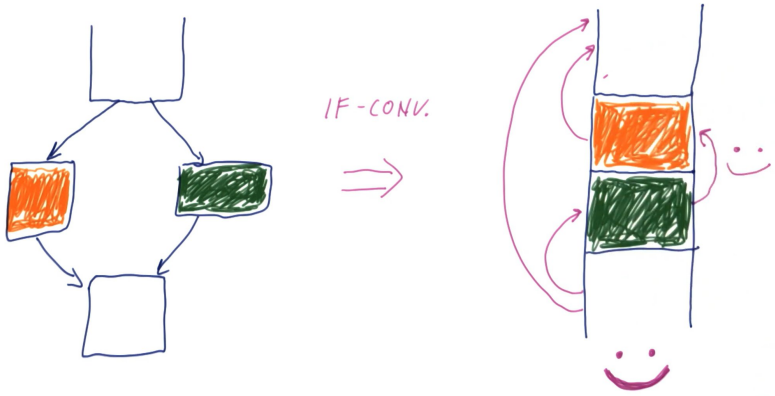
```
LW  R1, 0(R2)
STALL
ADD R1, R1, R3
SW  R1, 0(R2)
LW  R1, 0(R4)
STALL
ADD R1, R1, R5
SW  R1, 0(R4)
```

8 Cycles

```
LW  R1, 0(R2)
LW  R2, 0(R4)
ADD R1, R1, R3
SW  R1, 0(R2)
ADD R1, R2, R5
SW  R1, 0(R4)
```

6 Cycles

Scheduling and IF Conversion



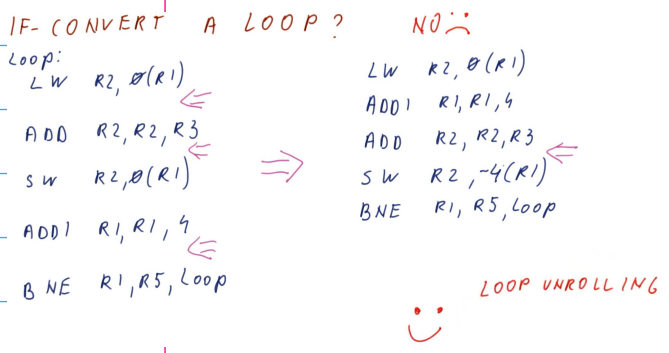
Before IF conversion, we have oppurtunity to shuffle instns only inside boxes, but if we do if conversion,

* We get lot more oppurtunities for filling stall cycles, so we can make better schedule for a processor.

* It also helps with branch prediction, because it helps with branches (removes some branches).

**So the idea of instn scheduling is that , we want to find instns that can be done in place of stalls, thus avoiding too many stalls and improve performance.

If_Convert a loop?



We have seen that we can if convert if elses A similar technique would be very useful for loops. But can we if-convert loops ?

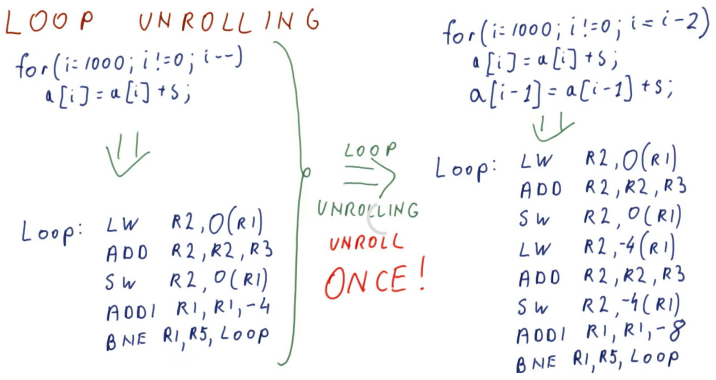
Lets go back to array example, where we get an array element, add something and store back, then inc the pointer until some end of array.

Say every instn takes 2 cycles to do, so we have stalls for raw dependencies. With scheduling, we have eliminated 2 stall cycles, so we have only one remaining stall.

So what if conversion does is it eliminates the branch at the end by predication, but we have seen that predication is not so suitable for loops. SO it allows the nxt iteration of the loop to be done and finally eliminate branch, and select output of any of the paths. Everytime we have a new iteration we would introduce a new predicate, and every new iteration would introduce yet another predicate, and we simply can't deal with so many predicates, so many things that get done only if the predicate is true. SO if we are exe a million iteration loop sometimes and sometimes only one iter, then almost a million iters would have to be just predicated out, which would be very very slow.

We would like to do something like if conversion for "loops", to allow us to find instns outside the iteration and reschedule them.

That is LOOP UNROLLING.



Given is example of unrolling once. Now we have half as many iteration, with each iteration doing twice as much work compared to before.

So to unroll,

*we copy the instns. twice and adjust the offset to point to nxt element of the array on the later copy.

*we need todo the loop counter update and bne instn at last, but for loop counter update we move index of array by two elements instead of one.

Loop Unrolling Benifits : #Instns ⬇

As can be seen from the above example, for doing 1000 iters , we would:

* Have 1000*5 = 5000 instns before unroll
* Have 500*8 = 4000 instn after unroll

We did this by eliminating "Loopoing Overhead". The last ADDI and BNE are called looping overhead. These instns help us go to nxt iter.

From Iron's Law, the Execution time = #Instns * CPI * T_clk

Here we are reducing execution time by reducing the #instns.
The smaller the loop, the more we benifit from removing loop overhead.
But lets look at what happens to the CPI.

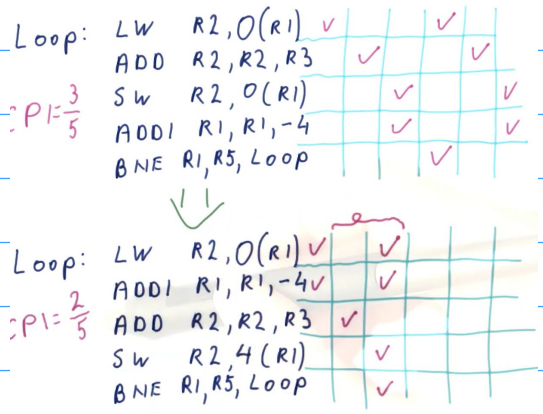
Loop Unrolling Benifits : CPI ⬇

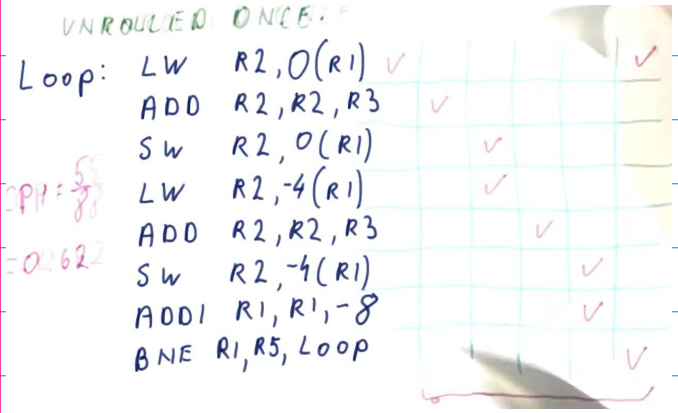
Lets say we have a 4 issue in-order processor, meaning it can see only nxt 4 instns to see what can be executed together. & has perfect Branch predictor. So lets see how this program executes. Also with scheduling.

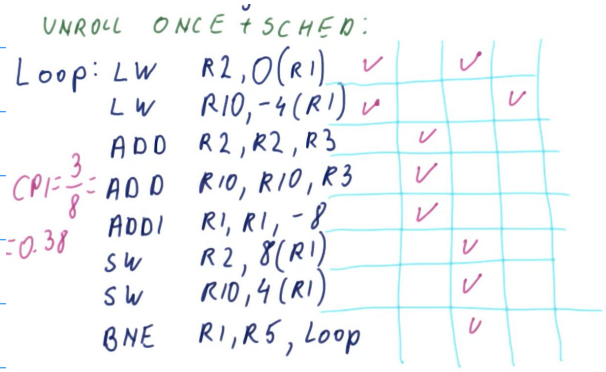
The only freedom we have to reorder is with the ADDI, because BNE needs to be at the end, lw, add, and sw needs to be in that exact order.

It can be seen that CPI improves significantly with scheduling.

Note that we can do LW without finishing BNE because of perfect BP.





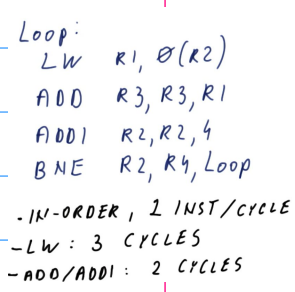


Note that our CPI is slightly reduced but our instns also reduced.

We can't do 2 loads at same time as we already did 4 instns in that C

Unrolling gives us more stuff that we can reorder, if we had unrolled thrice, we would had 4 loads that we can proceed with in same C, so the more we unroll the more parallel our code becomes. As we can see with unrolling we have slightly improved the CPI.

Loop Unrolling Quiz



AFTER SCHEDULING
____ CYCLES/1000 ITERATION

AFTER UNROLLING ONCE AND SCHEDULING
____ CYCLES/1000 ITERATION

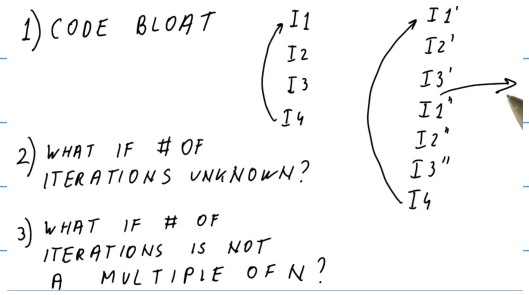
LW	LW	LW R1,0(R2)
✕	ADDI	LW R7,4(R2)
✕	✕	ADDI R2,R2,8
ADD	ADD	ADD R3,R3,R1
ADDI	BNE	ADD R8,R8,R7
✕		BNE R2,R4,LOOP
BNE		ADD R3,R3,R8
7000	5000	6*500+1 ~ 3000

Unrolling Downside ?

Code size is increasing proportionally to # of unrollings

What if iterations are unknown like in a while loop

What if # of iterations



Function call inlining

Function calls:

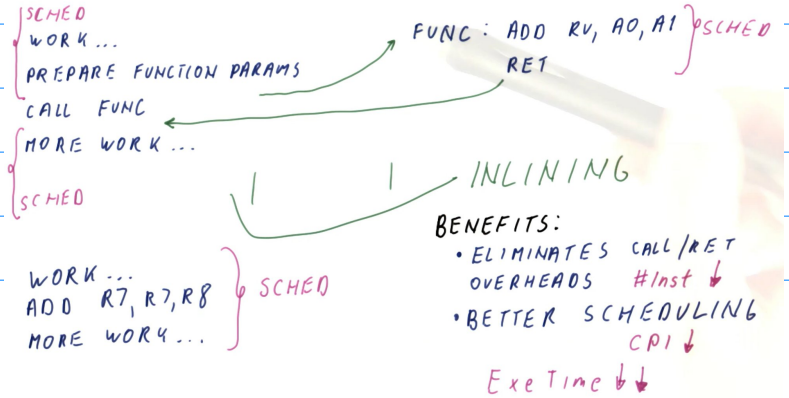
We do some instns
Then we prepare the func params (like put them in the right regs according to the calling convention mentioned by the ABI)
Call a func : Branch
Return from func
Do rest of the instns

We can avoid the overheadds of calling a function a returning from it by:

Do some instns
Do the work of the function (plugged here ,no need to prepare args, we put results eactly in which reg we need it)
Do rest of instns

Benifits:

* Calling overheads like 2 jumps in instn flow
* Preparing for func call, according to calling convention
* The code in the func. needs to pop params from the stack and so on
* After func returns we need to copy that value into required regs
* Scheduling becomes better, just like if-conversion did \
* Smaller the func . more benift from both :
  #instn count ⬇
  CPI ⬇

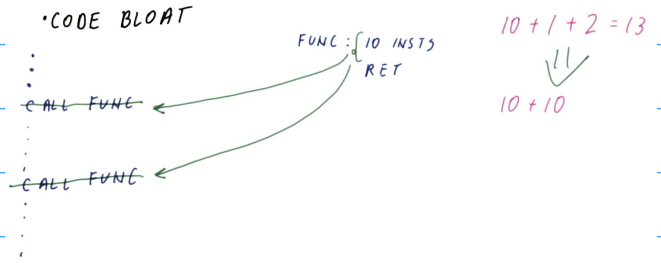


Function Call Inlining Downside

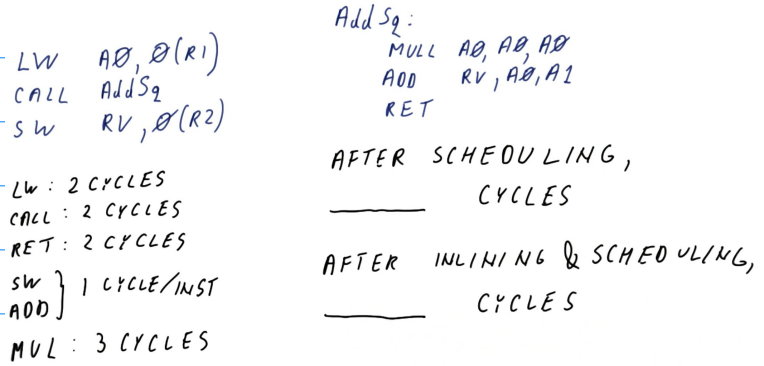
Just like Loop Unrolling, the downsides are:

Code Bloat: everytime we call func, we replace it with same instns

So, 10Instns 1 Ret and 2 calls (total 13) will become 20 if we do inlin.



Function call inlining quiz



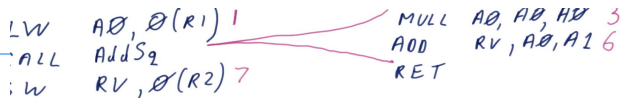
No scheduling possible here.

So,



Even after inlining, scheduling is not gonna help us.

So,

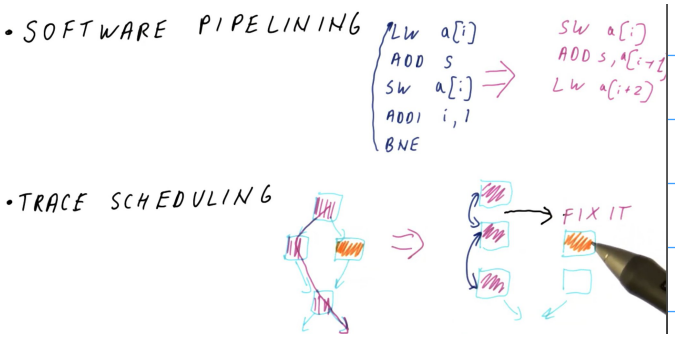


We have reduced exe time from 10 to 7 cycles here, with inlining.

Other IPC enhancing compiler stuff

The first is Software pipelining: treat each instn as a stage in pipeline and then schedule the code.

Another is Trace scheduling: We can think of it as if-conversion on steroids. We identify common path in the code and schedule it . Then we implement some checks to find if we are executing the wrong path, if so, we fix the damage(undo the effects of instns that we shouldn't have done, because we reordered) and move to correct path. The catch here is we need the common path to be common case. So we save time overall.



These techniques are well explained in an advanced compiler class. This lesson introduced some of the more advanced compiler techniques, that help produce programs that work better with the branch predicted OoO multiple instn per cycle processors that we have today. In the nxt lesson we will look at a type of processor that will simplifies its hardware by relying more on such compiler support.

VLIW

Intro

In this lesson we will learn about VLIW, and explicitly, parallel processors such as intel's itanium, unlike their OoO cousins, these processors do not try to identify ILP on their own, instead they just do what the compiler tells them to do.

Superscalar vs VLIW

We have seen that a superscalar processor will try to exe more than one instn per cycle. VLIW processors are trying to achieve the same amount of work per cycle but in a diff way. Let's compare them:

	OoO Superscalar	InOrder superscalar	VLIW
Instn/Cycle	≤N	≤N	One large instn. =work as N small Is
How do we find dependent inst	See >>N Instns	Look at nxt N Is in program order	Just do the nxt large instn.
Hardware Cost	Expensive	Less expensive	Even less expensive
Help from compiler	Compiler can help	Needs Help	Completely depends on compiler

Inst/Cycle: An N- issue OoO Superscalar, does upto N instns / Cycle
Similarly InOrder Superscalar
VLIW is diff in the sense that it tries to execute one large instn that is equal to work of N "Normal instns would do".

How do we find independent instns:

OoO looks at much larger window (>>N) of instns to find independent insts
So it fetches, decodes and has more than N instns , and among these we try to find the N that it needs to exe.
In contrast, an InOrder proc. only looks at nxt N instns in program order.
It doesn't maintain huge instn window among which it executes, instead it just tries to see the nxt N instns and tries to see how many of them can be executed in parallel in program order.
A VLIW, doesn't even try to see nxt N independent instns, it just does the nxt (large) instn., so it behaves like a non-superscalar processor that executes instns in order, its just that each of these instns are describing a lot of work.

Costs:

OoO needs to look as >>n instns and find N independent instns out of those is really expensive.
inOrder Superscalar does a lot less work,
(cause it looks at only nxt N instns in program order,) so needs lot less hardware compared to OoO parts.
VLIW is even less expensive, for doing same amount of work per cycle, assuming we can find that work (because now we are even simpler in finding that work).

Help from Compiler:

An OoO superscalar does pretty well even without any help from compiler, but compiler can help to improve the performance.
InOrder superscalar is more dependent on the compiler, if the compiler doesn't do anything to put indepenent instns consecutively, then it will not have good performance(it will lag significantly compared to OoO with same issue width). SO it really needs help from compiler.
VLIW Completely depends on compiler to produce performance.If we don't have a good compiler, the VLIW processor will fail miserably in performance.

SO OoO superscalar is trying to do hardware intensive " I dont need much help from anybody " But it sacrifices hardware to do that. An In order superscalar tries to reduce hardware cost , but still can execute the same program that an OoO SS could execute, and in some programs it really needs compiler help for good performance. VLIW completely depends on compiler to provide & express the parallelism it needs to do work, but in exchange it gives us much lower costs.

Superscalar vs OoO Quiz

Lets say we have VLIW and a OoO , say OoO executes 32 bit instns, while VLIW executes 128 bit instns , where each VLIW instn specifies 4 operations, (each operation is same as one instn worth or work in OoO).

If we have a program size of 4000B in length for OoO processor,

Program size is between _____ and _____ Bytes.

4000B if we manage to package 4 instns into each VLIW

In worst case say if all these instns depend on each other, then the size becomes 16000 B because only one op possible in one VLIW and 3 will be no-ops (we can't fit dependent ops in same VLIW instn.).

VLIW Good and the Bad

Good	Bad
*Compiler does all the hard work, because compiler only does work once and then the program is run multiple times, compiler has plenty of time to find a good schedule. In an OoO proc. we have very littly time to come up with a good schedule, because any time we spend figuring out the schedule, is the time we are adding to the program execution time	* Latencies of the instns are not always the same, (the compiler has to assume some sort of latency in order to schedule code well, but sometimes latencies can't be determined exactly (say Ld has cache miss and we assumed hit)
* Simpler hardare than a comparable OoO	*Irregular applications (almost every application that has to do lot of decision making is irregular like AI applications, apps that work on pointers or pointer intensive structures are very hard for the compiler to figure out)
* Can be more energy efficient than an OoO	* Code bloat issue, lot of NOPS
* Works well on Loops, and "regular" code	

Regular code is code like sweeping through arrays, matrix mul, and other things that the compiler can easily figure out and schedule well around it.

VLIW Processors backward compatibility

Lets see if a program compiled for one VLIW works on a better VLIW proc.

QUIZ

Say we have a simple VLIW, that does 64 bit instns(2ops/cycle) and fetches, decodes, and executes (1 inst/ cylce)

Say we have a better VLIW, that can do 4ops/cycle and we use the same code and try to do 2 64-b instns per cycle. Is this a VLIW??

Solution

No, not really , because when code is compiled for simple VLIW, we have 2 independent operations in single instn, but when we do two such instns per cycle, its not guarenteed to be independent all 4 operations, so this "better" processor has to do some dependence checking, making it more of an normal superscalar and not a VLIW.

VLIW Instructions

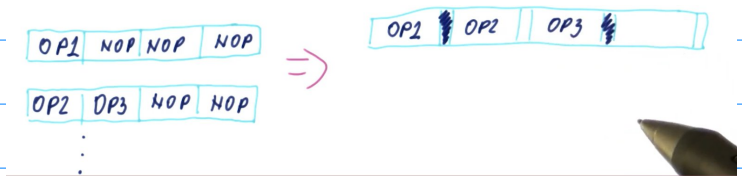
The ISA has all the normal ISA opcodes.

Has support for full predication (or atleast very extensive pred support) This is because , it relies on the compiler to expose parallelism, one of the ways compiler does that is scheduling instns

Lot of architectural registers (when we are rearranging instns we need to put things in other instns , we have seen, so to help compiler do instn scheduling, we need more regs, and we provide a lot)

Branch Hints (compiler hints HW , if it thinks the instn is a branch) this helps the Branch Predictors

VLIW Instn Compaction, basically we put delimiters to tell if the instn is over and pack more instns into single instn, eliminating NOPS and thus saving code space.



VLIW Examples

The most famoud example on how to NOT do a VLIW is intel's Itanium proc.

It had tons of ISA features that helped ompiler schedule code well etc
As a result HW became so complicated,
It no longer needed to check dependencies berween instructions, but it had lot of other bells and whistles to support ISA features. Its probably the most complicated proc. intel has ever built(as of 2015).

Itanium was still not so great on irregular code

Another example where VLIW is often used is DSP processors. Because it has regular loops and lots of iterations (typically floating point)
Excellent Performance
Very Energy efficient

So if we try to run all types of codes on VLIW, we run into problems,

but if we do for only some types of codes (where compiler can do well)

then we benifit a lot.

VLIW Target Market Quiz

VLIW is best for:

* An application that adds up many numbers

* An application that figures out best path in a maze

* An application that counts elements in a linked list.

Figuring out best schedule for adding many numbers is doable for compiler.

In contrast, figuring out the best path in a maze, typically in volves a lot of decisions and a lot of if conversions so its very inefficient, because a lot of work we are gonna do is gonna be predicated out.

Counting elements in a linked list is gonna involve lot of load instns some will be hits and some will be misses, so our compiler schefule will be greatly effected by cache misses.

So adding is best scenerio for VLIW, followed by counting elements, and

It is unlikely to do well on figuring best path in a maze.

In this lesson we have discussed VLIW processors and their itanium cousins , we have learnt the pros and cons of these processors, and some of the reasons for itanuym's high expectations, and eventual demise.

This lesson concludes the 1st part of HPCA course,

We learnt how modern processor core works

In the nxt part we will learn how to get good performance even though the main memory is really.. slow.. and way too small to fit the programs we want to run.

After that, the final part of the course, is about putting multiple proc. cores together to get even more performance and energy efficiency.

Caches Intro

In this lesson we will be reviewing caches, we already know most of these, but it is extremely important to unerstand the basics of caches before we move on to advanced topics like virtual memory and other more advanced caching topics , so this lesson has much more detail than your typical review type lesson.

Locality Principle

To understand caches, we must first understand the concept of locality, which is also called the locality pronciple, which says that "

Things that will happen soon are likely to be close to things that just happened"

This means that if we know something about past behaviour, we we are likely to be able to guess, whats gonna happen soon , we have already used this locality principle for example when we did branch prediction, now we will use it for caches.

Locality Quiz

Which of these are not good examples of locality (given conclusions)?

* Rained 3 times today ⇒ its likely to rain today again
* We eat at 6pm everyday last week ⇒ probably we eat @6pm this week
* It was new yr's eve yesterday ⇒ probably it will be new yr's eve today

First two examples are good examples of locality

3rd is not because it doesn't have locality, (it is one of those things that do not have much locality) , if its yesterday we are sure it will not be today, its kind of opposite to locality principle.

Some things do have locality property & some things do not.

Memory references

In computer architecture we are intrested in locality property of memory references.

If we know that the processor has accesed adress "x" recently, the locality principle says it is likely that proc accesses it again in the near future & it is likely to access addresses close to "x" too.

First is called temporal locality, and the nxt is called spatial locality

Temporal locality quiz

```
int sum=0;
for( int j=0; j<1000; j++)
        sum = summ + arr[j];
```

Consider the given code snippet, which of these mem locations has temporal locality in this code? MSQ

* j
* sum
* elements of arr

Temp locality says that, once we access the location, we are likely to acces it again.

For the variable j, we are using it to check if its equal to zero, to inc it, as well as to see if j<1000 and to access the elements of the array in every iter of the loop. So j has temporal locality.

Similarly for sum , we will be accessing it a lot .

Elements of the array? not really because we are not accessing it again once we access it. Only once we are accessing it in the loop, thats the only time we are accessing it in this loop.

Quiz-2

Which of the mem locations has spatial locality in this code ? MSQ

* j
* sum
* Elements of the array

Spatial locality says once we access a mem location, we are likely to access nearby mem locations soon.

For the var j, we keep reaccessing j, but we don't know whats close to j, so we can't really say we have spatial locality for j, there is a lot of temporal locality but not spatial locality, same goes for sum.

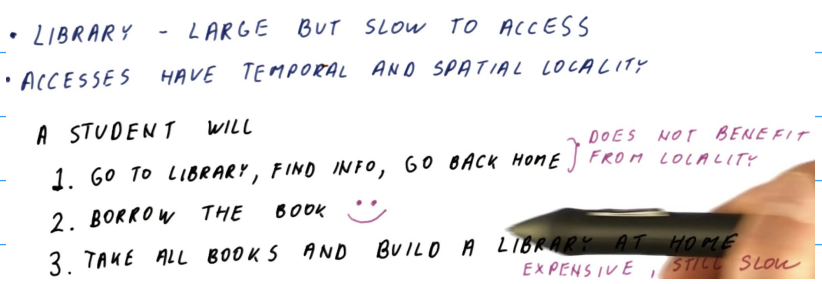
We are accessing sum very often but we are not sure if we are accessing a anything that is close to sum.

For elements of array , we have seen that there's no temporal locality, each element if only accessed once in this loop, however there is lot of spatial locality, once we access arr[0] we are likely to access arr[1], then arr[2] and so on. So it is true that once we access a mem location in this array, we are likely to access nearby locations soon. So this var definitely has spatial locality.

It turns out that the way this program is compiled, usually means that there will be some spatial locality between j and sum ,they are likely to be allocated by the compiler on the stack near each other . So when we access sum and then j, that might be an example of spatial locality.

Locality and memory accesses

So now that we know what locality is, lets see how we can use that to improve memory accesses.



People usually choose the second option, because, its a good treade off between being able to find information quickly and not being overwhelmed by the quantity of the information we have brought in.

When we have a processor , that needs to access the main memory, we will use the same approach, instead of oging to main memory for every single memory location, we will bring the content of memory locations that we are intrested in closer to us, but also we will try not to bring to much memory close to us(like inthe 3rd option) because that will result in a very slow structure, so we will again have slow accesses. So what we want is something small that keeps only the info that we really intrested in. And that small repository of information, is called a cache when we are talking about memory accesses.

Cache Quiz

We know that main mem is large and slow to access compared to proc speed

And we know there is lot of spatial and temporal locality in our data accesses (that the case with most programs) for a access to mem location what should we do.

* GOto main memory for every access
* Have small mem inside proc core and bring stuff we access there
* Have a huge mem chip aside proc chip and we bring everything as we read from main mem, there

THe options are very similar to library example.

Second approach is similar to Caching

the third approach will save us some time, but still it will be slower because of its size.

So caches are exploiting the spatial and temporal locality, as well as overcoming the problem of having a slow and large main mem.

Cache Lookups

So now that we know that cache is a small mem inside our proc. where we are trying to find data before we are going to the main mem. Lets take some time and recap what we need out of a cache.

*We need it to be fast ⇒ it needs to be small

*Because its small, not everything will fit,

* So when a proc. want to access mem, we can have a

    * Cache hit, Found it in the Cache (the mem location we are looking for was already brought from the mem to the cache)
    * Cache miss, because its small there can be misses, access the slow memory and then copy this into cache

We need to have cache misses, because that how really we bring stuff into cache. The more locality we have, the more hits we will have. Cache hits are fast. And misses are sloww, and we need misses to be rare, thats how we want to design our caches.

Cache Performance

Lets see what are characteristics of a good cache.

We want our system to have a good AMAT (average memory access time) it is the time as seen by the processor , the access time to the mem.

AMAT = HIT_TIME + MISS_RATE * MISS PENALITY

Lower hit time means, small and fast cache; Lower miss rate means we need large and/or smart cache; Miss penality is simply main mem access T, this is going to be very large , 10s or even 100s of cycles, so really

when we are designing caches, we are trying to balance hit time and miss rate. Small& Fast vs. Large & Intelligent and Intelligent means slightly slower.

Sometimes we talk about miss time, i.e overall time it takes to have a cache miss.

MISS TIME = HIT TIME + MISS PENALITY

AMAT = (1-MISS_RATE)* HIT_TIME + MISS_RATE*MISS_TIME

Hit time Quiz

In a well designed cache, which of the following is true

* Hit time < Miss time — hitT << Miss T
* Hit time > Miss penality — False
* Hit time == Miss penality — False
* Miss time > Miss penality — Miss time ~= MP

If miss penality is == hit time, then we dont need a cache, we would go to mem each time. Miss time is hit time+ misspenality. So last option is true for every cache not just for a well designed cache.

Miss Rate Quiz

Hit rate = 1- Miss rate

Which of the following is true for a well designed cache

* Hit rate > Miss rate — HR >> MR
* Hit rate < Miss rate
* Hit rate == Miss rate
* Hit rate is almost 1 — HR → 1
* Miss rate is almost 1

Cache sizes in real processors

There is a complication here, it is that, the modern processors have several caches, so lets look at caches that we call L1, which are the ones that directly servies rd/wr requests from the processor. So in recent processors, level1 caches have typical sizes 16kB to 64kB, this is large enough to get 90% hit rate. Yet small enough to access in 1-3 Cycles.

Cache Organization

Now that we know what is the purpose of the cache and know how big it typically is, lets see how it is organized internally. The questions we need to ask is

* how do we determine we are hit or a miss (once the proc. gives an address we need to determine if its in the cache or not and we need to do that quickly).

* how do we determine what to kick out (when we out of room in cache, and we hace a cache miss, so we need to bring something in)

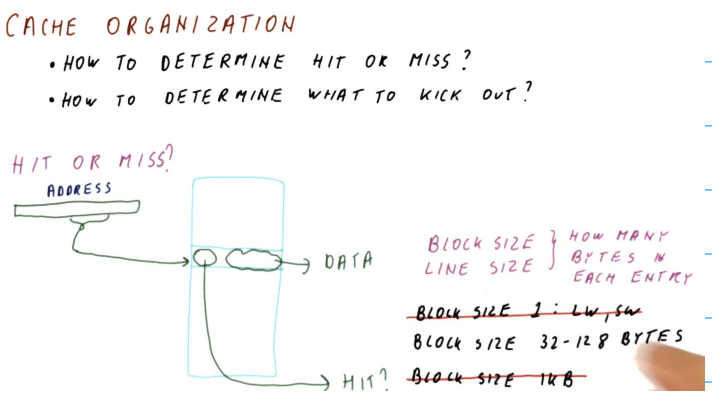
TO determine if its a hit or a miss, we need to do determine it fast, so we can have a table which can be indexed into using some bits of the addr., and the table's entry tells us what we have in the cache, if its the same as what we need, then it gives us that data.

Conceptually cache is a table that we index into using some bits of the addr., so inside each entry, we need some stuff that can tell us if its a hit, and the rest of that is the data that we want if its a hit. How much data we hae in each cache entry ? is called Block size or the line size. Having a block size of one creates several problems, (so in this case, each address (of data) maps to diff entry in the cache,) usually proc. requests one word (4B) in Lw and Sw instns., so for a single access, we need to lookup 4 diff cache locations, which complicates and slows down cache a lot, so we definitely dont want block size to be this small. We want our block size to be atleast as large as largest single access we make to cache, and better if its larger because when we do lw or sw, its better we find the data in same cache block without going to mem again The nxt thing we are going to consider when deciding the block size is spatial locality. When we miss in cache , we bring in an entire block worth of data into cache from mem.

If there's no spatial locality, we just want to bring in what is currently needed, But if we have spatial locality, we want to bring in more than whats needed in this access. So typically, block sizes of 32 - 128B work well , both from the perspective of they are larger than word size, & they capture much of the spatial loacality that exists in programs.

Now lets look at block sizes of 1kB, in that case what happens is we fetch a lot of data from mem in case of a miss, and if there is not good locality, a lot of that data will not be used, (Remember the cache is only 16-64kB,) so if we are bringing this size of a block, we are consuming significant part of available space, and we are doing it with data that probably are not going to be used much, So for level1 caches, we dont want block sizes that is too large.

So block sizes of 32-128B are a balance .



Block size Quiz

Consider a cache that is 32kB in size and 64B block size.

Say a program accesses variable x1, x2, ... xN, these are scalar vars (not part of an array) Lets assume the program has lots of temporal locality when accessing these vars, but no spatial locality, The q is what is the largest N that still results in a high hit rate?

Sol : N is 512

If we have ≤ 512 vars, then each one of them will have its own cache block (although each var is just 4 B), and is not gonna hurt in terms of hit rate. If we need more than 512 vars, we need some spatial locality, so that a block will have not just one var but some of others as well.

Cache block start address

Now that we know when we have a cache miss, we fetch entire block worth of memory, lets talk about where can these blocks begin in memory.

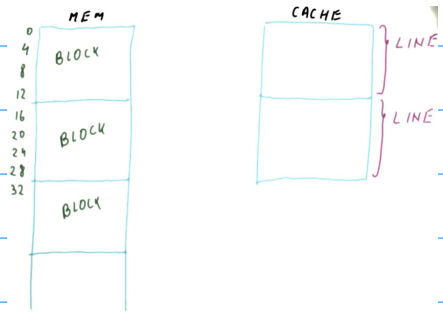
To avoid complications and problems of duplication, we use block aligned adresses. So that any byte we need can be found only in one uniq particular location (not in many) . This also allows us to index into cache using some bits of the addr.

So for all reasonable caches, we will only consider block aligned address for fetching into cache.

Blocks in Cache & Memory

Now that we know the cache blocks need to be at aligned mem addresses, lets see how memmory and cache look like in terms of blocks.

Say our block size is 16B, its also the same as line size, just that its called a line in cache and a block if its in mem.



Line size Quiz

Which of these line sizes are not good line sizes in a 2KB cache

*1B *32B *48B *64B *1kB

1B is not a good size because it doesn't explioit spatial locality,

32B is a good size, it exploits spatial locality, & it is power of 2, given the addr, we can easily locate which block it belongs to

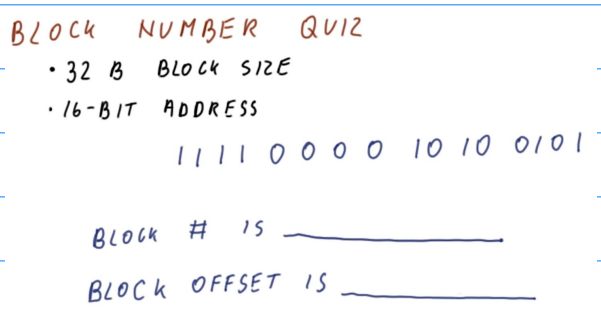
48B is not good size, not because of its size, but because its not a power of 2, so if we are given an addr. and we need to find which block it belongs to , we need to do some computation (div by 48) which is much more complex than div by power of 2.

64B is again a reasonable block size, its bigger than 32, but also a lot of them will fit in a 2KB size

1Kb is not a good size, because only 2 of them will fit in 2KB cache

What we need is, it should be power of 2, expliot spatial locality, but still many of them should fit in a given cache size.

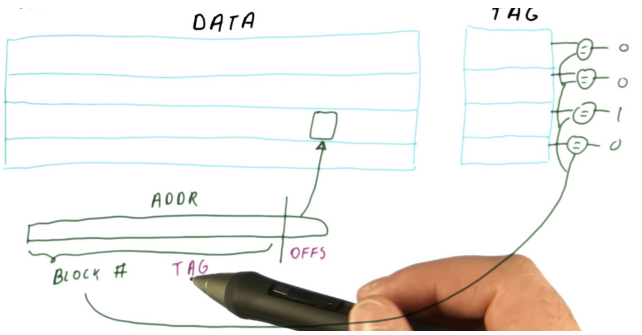
Block Offset and Block Number



offset is 5 and block # is 785

Cache Tags

Now that we know how to find block offset and block number, lets talk cache tags that help us determine which block do we have in the cache. Cache in this case has 4 #of lines. Cache Tag is not always same as block number, but in this case it is.



Cache Tags Quiz

The tag always contains: (MSQ: multiple select question)

* The entire addr of the first byte

* At least 1 bit from the block offset

* At lease 1 bit from the block number

* Some bits from the offset and some bits from the block #

The tag just need to contain the block# part not the offset

It contains atleast 1 bit from the block number, we have seen that tag contains all the bits of block number, in general it can be fewer.

Valid Bit

Another piece of information, the cache maintains about each of its blocks is, valid bit. Say we have uninitialized / invalid data in cache, and we incorrectly see that we have a hit, so for that sake we have valid bit. We valid bits , we remove the problem of initializing the data.

When we boot up the processor, we set all valid bits to zero, and when we bring stuff from memory, put data in cache, set the tag bits and also we will set the valid bit.

Types of Caches

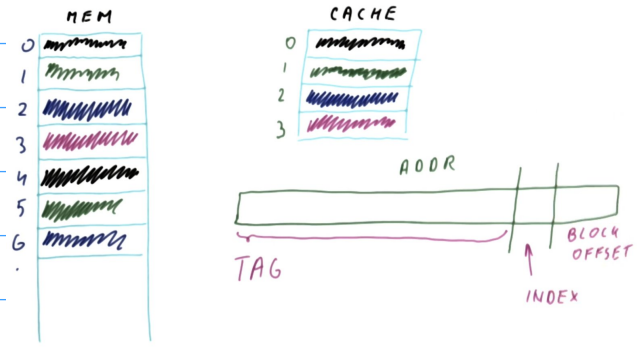
There are several types of caches, when it comes to which blocks can be placed where in the cache. The cache examples we have seen until now are fully associative caches.

The opposite is direct mapped cache, different blocks ofcourse can map to different lines, but a particular block can go only to one place(line).

Then there is middle ground between them, which is called a set associative cache, there are N lines where a block can be, where N is 1<N<#of lines. So we can view Direct mapped cache is a type of set-assos. where N is 1 and Fully associative is type of set associative where N is equal to #of lines in that cache. But we typically have N relatively small

Direct mapped cache.

We have seen a glimpse of fully associative caches, now lets go from the other direction which is direct mapped caches.



Direct mapped caches Upside and Downside

Fast , because we only need to look in one place, (if we were to look at more than one place, we need to read out all that locations, and then figure out which one of them has it and so on) so less hit T

Cheap , because we only need one tag comparator on every access to cache

Energy efficient, because only one comparision

Downsides are

* The block must go only in one place , to know why its a problem, consider proc. accessing A & B alternately, which both map to same line in cache, then we would have miss everytime. SO the problem here is conflicts. So they increase miss rate.

SO we do have fast hit times, but also we are having higher miss rates.

Direct mapped Cache Quiz

16KB , 256B blocks cache, which of these addresses conflict with 0x12345678, 0x_____? , MSQ

* 12345677
* 11335577
* 11115678
* 12341666

As its 256B block, we have 64 lines in total, so last 6 bits (after 8b) of addr. tell which line to go. so any line that has xx01_0110 will be conflicting. so answer is third and last will conflict. First will not because its in the same block.

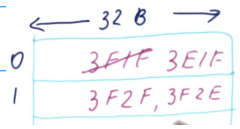
Direct mapped cache quiz

say we have 8 line dir-mapped cache with each line being 32B. Now that the proc. issues requests as:

*3F1F *3F2F *3F2E *3E1F

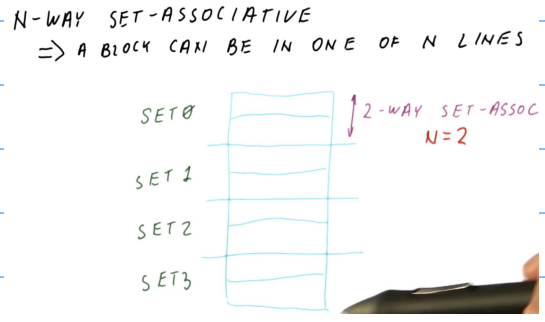
What is the state of the cache after each access.

Sol: So lsb 5 bits tell offset, then nxt 3 bits (R2L) tell line#.
So, 3F1F goes to line#0
3F2F goes to line#1
3F2E goes to line#1, but the tag is same so it can be found
3E1F goes to line#0, but the tag is diff so it replaces line

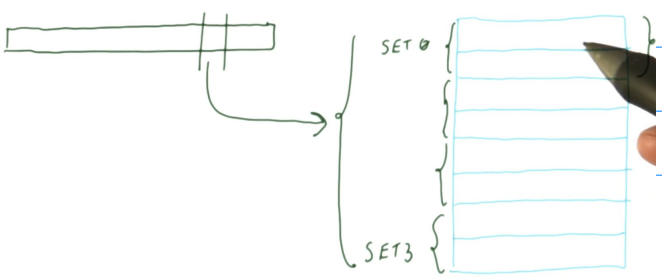


Set Associative Caches

Now lets look at set assosiative caches. We say a cache is N-way set associative, when a particular block can be one of N lines. Now the overall cache is divided into "Sets". We use some bits of block's addr. to tell us which set it needs to go to. With in a set there could be a no.of lines, that can contain a block. That no.of lines is the associativity of the block.



Offset, index & tag for set associative cache



Set associative quiz

We have 2 way set associative cache, with 4 sets, just like shown above, each line is 32B, what is the final content if the addresses are

0xf303 0xf503 0xf563 0xef63

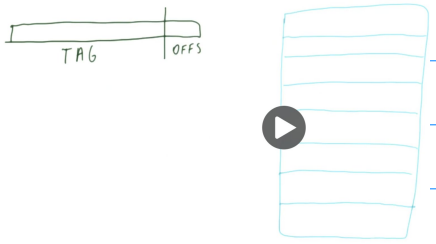
Sol: lsb 5 bits are for block offset, then nxt two bits decide set,

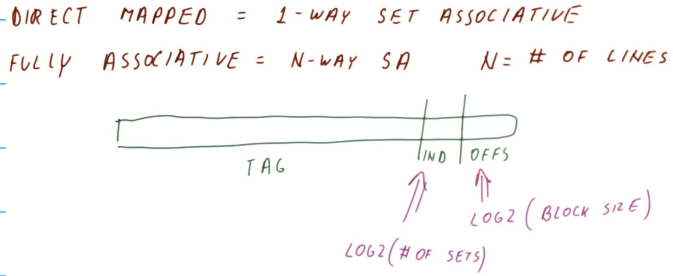
f303 goes into set0, f503 goes into set0 bottom place

f563 goes into set3, ef63 goes into set3 bottom place

Fully associative Cache

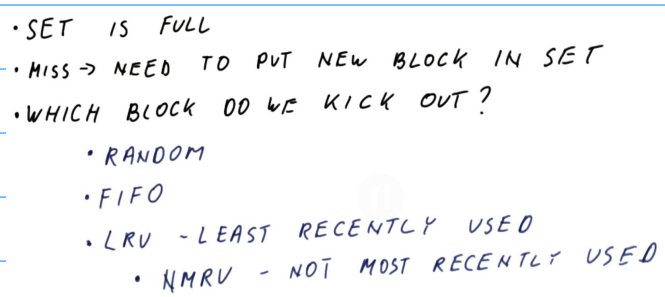
Any block can map to any of these lines. No index bits.





First determine #of Offset bits only then we can determine the nxt that many #of bits will be index bits.

Cache Replacement

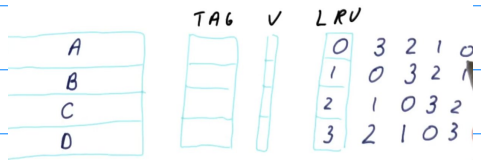


It turns out that the LRU is a really good policy, so there are several policies that are trying to approximate it because this is not easy to do.

One such policy is NMRU, which tracks the most recently used block, and picks randomly among the remaining blocks. So its like dont pick the block that was recently used.

Implementing the LRU

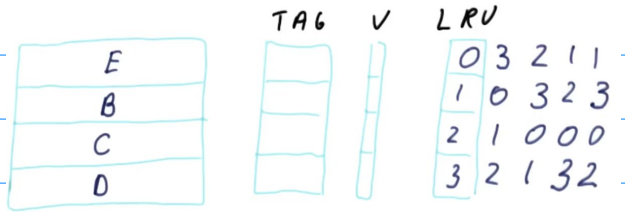
LRU works really well because it exploits locality. Say we have a 4 way set associative cache, So along with valid and tags, we are going to maintain LRU counters for each line, these counters take the values that correspond to size of the set. So lets initialize them to be 0, 1, 2 & 3. They all has to be different, and we will see how that is maintained. When we want to replace, we will replace the line whose counter is currently zero, because thats gonna be the LRU block.



When we place A, its gonna become 3012, when B 2301, when C 1230, and when D is placed LRU entries become back to 0123. (Every other greater cntr is dec each time something is placed .)

So when we place E we replace A and make the counters 3012.

Now say B is reacessed, now this B counter becomes 3, and the rest dec. So it becomes 2301. What happens when B is accessed again, counters stay same. What happens when we access D? D has value 1 that will become 3. And all the counters whose value was above 1 will be made to dec. the values that are below 1 stay the same, so it becomes 1203. Now say B is accessed. Now it becomes 3 and 3 becomes 2, rest stay same, so seq is 1302. After every step we can verify all counters ahve diff values.



So for highly associative caches, like 32 way, we need to have 32 #of 5bit counters "for each set".

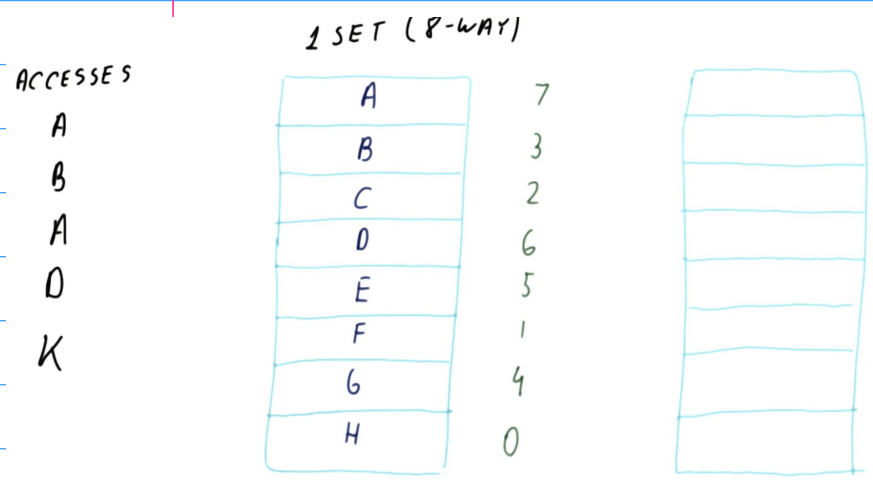
COST: N-WAY SA → N LOG2(N)-BIT COUNTERS

ENERGY: CHANGE N COUNTERS ON EACH ACCESS (EVEN HITS)

We have high cost as well as energy for counters, even on cache hits, which is very often.

So we have LRU approximations that try to keep fewer counters and do fewer updates (to save energy) on cache hits.

LRU Quiz



A7	A7	A6	A7	A6	A5
B3	B3	B7	B6	B5	B4
C2	C2	C2	C2	C2	C1
D6	D6	D5	D5	D7	D6
E5	E5	E4	E4	E4	E3
F1	F1	F1	F1	F1	F0
G4	G4	G3	G3	G3	G2
H0	H0	H0	H0	H0	K7
	A	B	A	D	K

Write Policy

The final aspect of how caches work concerns write policy. There are really two parts of it, first allocate policy i.e Do we insert for blocks that we write. If we have a write miss, does the block is brought to the cache or not. According to this we have two types of caches,

* Write Allocate: brings the block that we write into the cache

* NO write allocate: doesn't bring block that we write into cache.

Most caches are write allocate, because there is some locality b/w reads and writes.

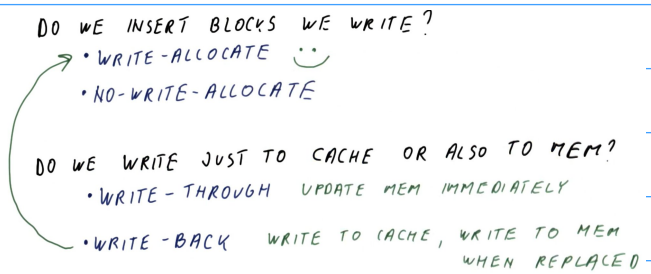
THe other aspect of write policy is , when we have a write do we write just to cache or also to mem. So based on this we can have two types of caches:

* Write through: update the mem immediately

* Write back : write to cache, and we only write to mem when the block is replaced from the cache.

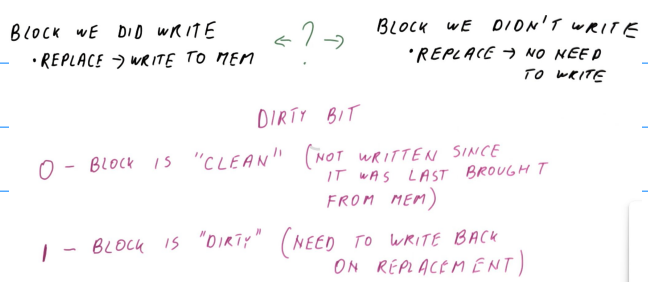
Write through is a very unpopular cache, we usually want to have a write back cache, because that way, writes(which have lot of locality) update the cache many times and update mem only when this block is replaced. THat avoids mem being overwhelmed with # of wr requests.

There is also some relation between "allocate or noallocate?" and "write through & write back" . If you have a write back cache, you also want to have a write allocate. Because you are saving writes to mem by only writing to the cache, if you have a write miss, you want to have future writes go to cache, so you do write allocate.



Write back caches

Lets look about what happens in a write back cache, write through is fairly simple, you just send writes to mem, and cache works just as we described it before. But for Write back caches, we can have a block, that we did write since we last brought it from the mem, in that case, then we replace the block, we need to write to mem, But also there can be block, that we didn't write since we brought from the mem, in that case, when we replace it we need not write to mem. How do we know if its written?? One choice is that we don't wanna know, everytime we replace the block we write it to mem, But unfortunately there is lot of readonly(from cache) stuff, so we would be wasting writes with same data as it was there in mem. SO in that case we choose to add a "dirty bit" to every block in the cache. This bit says if the block that was placed in the cache is written



Write back cache example

Say we have a simple direct mapped 4 entry cache, we have valid, tag , data and dirty bits. We don't need LRU counters because its not set associative. Say A, B & C are mapped to different sets in the cache. Initially we have all the valid bits as zero , so it doesn't matter what we have in the other fields of the cache. Say the processor has this seq.

wr A rd A rd B rd C wr C rd E

When writing A, we bring A into say first line, and make it valid, then, we put tag bits of A in the tag location. Because this is a write, as soon as the data is brought into cache, it will modify A, and the dirty bit will be set to 1.

Nxt the processor reads A, it sees tag matches A and its valid, so it reads (the fact that dirty bit is 1 doesn't change anything).

Nxt we have rd B, we dont find any cache entry that contains B, so we bring in B (i.e we didn't find matching tag in valid cache entries), say we being to line2. because its only a read, the dirty bit is 0.

Nxt we see its a miss, we bring in the block that contains C, put in cache make it valid, its a read, so dirty bit is 0.

Nxt we have write to C, we check if its a hit, and it is , we write only to the cache, and we change the dirty bit to 1. And now dirty bit for everyline simply indicates it been modified.

Say now we have read E, and say E maps to where A is . So we gonna check in first line if the tag matches, it doesn't so we have a miss, we dont directly bring in E and replace A, but because the dirty bit is 1, this line gets sent to mem as a write, and after that happens, we can then bring in E.





Write back cache quiz



Suppose we have a direct mapped cache, we shown a single entry in that cache, and say all the access we do will map to the same entry. Say valid is 0, D is 1 and tag is A initially. The processor does the following seq of accesses.

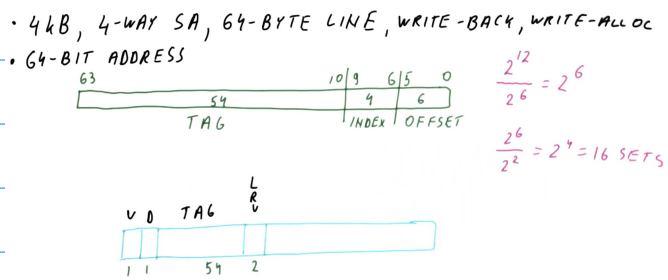
rd A rd B wr B rd C rd D wr D

After all these access what is the new state of the cache. Also how many cache misses do we have and how many write backs.

Sol: rd A will be a miss :: 1 0 A
rd B will be a miss :: 1 0 B
wr B will be a hit :: 1 1 B
rd C will be a miss :: 1 0 C but after we Wr Back B
rd D will be a miss :: 1 0 D
wr D will be a hit :: 1 1 D

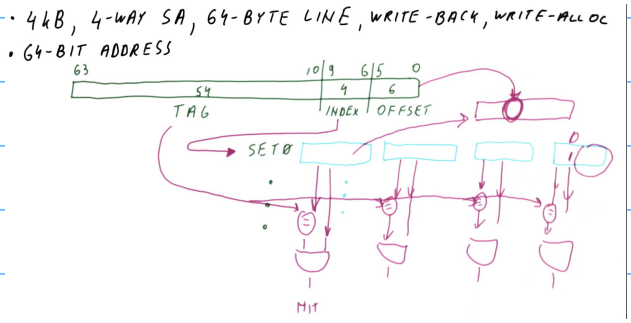
So solution is 1 1 D, 4 misses and one wrBack

Cache Summary



So for each 64B line, we have these 58 bits extra called overhead. SO actual cache size will be some extra memory than its actual data size.

So when we have a data req. , we see the index bits to tell which set the datea belongs to , in that set, we compare all the line's tag to tell which line has our data, if any has, we use offset to read the data that we need from that line, and if its a write, we modify it and we just set the dirty bit 1 (its faster to set it 1, instead of checking it if its 1, and only if not we set it) . In case if its a miss (we didn't get any tag match), then we see which line has LRU counter as 0, then check its dirty bit, if its 1, we write this data to mem and then put the new block here, but if the dirty bit is 0, we simply bring in the new block and update the tag and everything, we set the LRU counters accordingly, and then we supply data to the processor. So as we can see all the stuff we have been talking about here is kinda happens simultaneously, on every access all of this needs to happen.



Cache summary quiz

• 256 B, 32 B LINE, 2-WAY SA, WB, W-ALLOC
• 32-BIT ADDRESS, BITS NUMBERED 0 .. 31

TAG BITS ARE ____ .. ____ — Sol: 31 – 7

INDEX BITS ARE ____ .. ____ — 6 – 5

OFFSET BITS ARE ____ .. ____ — 4 – 0

LW 0x BCDE0000
LW 0x CDEF0000
SW 0x BCDE0000
SW 0x CDEF0004
SW 0x BCDE0000

MISSES IS ____

OF BLOCKS WRITTEN BACK TO MEMORY IS ____

1st access is miss, goes to set 0
2nd access is miss, goes to set 0
3rd access is hit, goes to set 0, and dirty bit set
4th acess is hit, goes to set 0, and dirty bit is set
5th acess is hit, goes to set 0, DB is set

Cache review Outro

In this lesson we have reviewed how caches work and what are the concerns and choices when designing caches. We will use this knowledge when designing most of the subsequent lessons in this course starting with virtual memory and advanced caches. And it will also be definitely be helpful in projects.